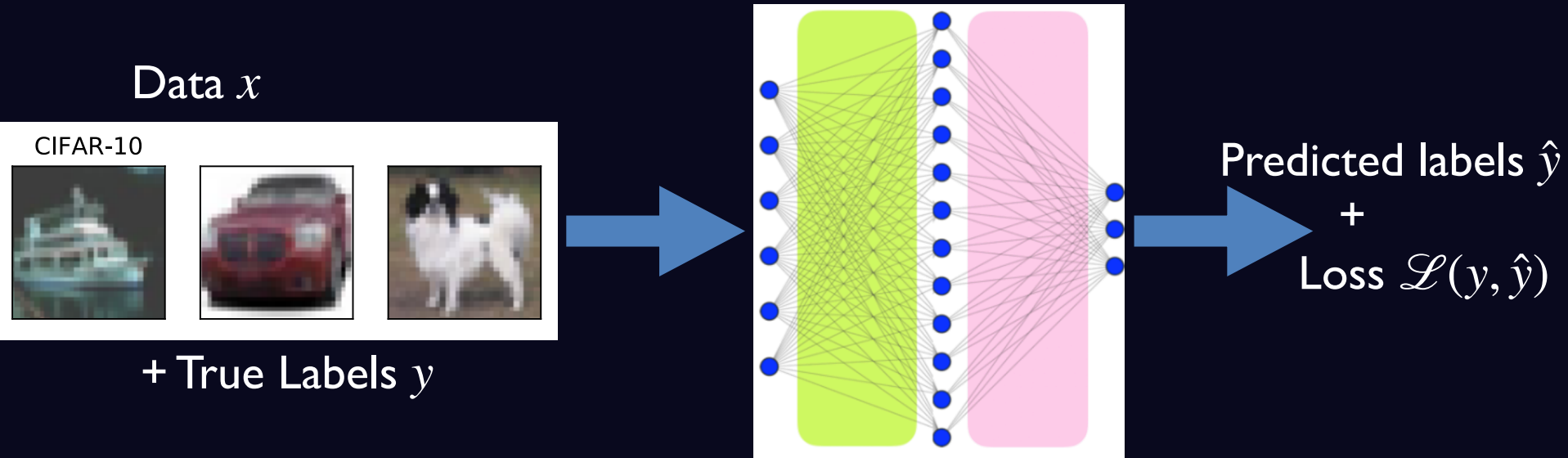


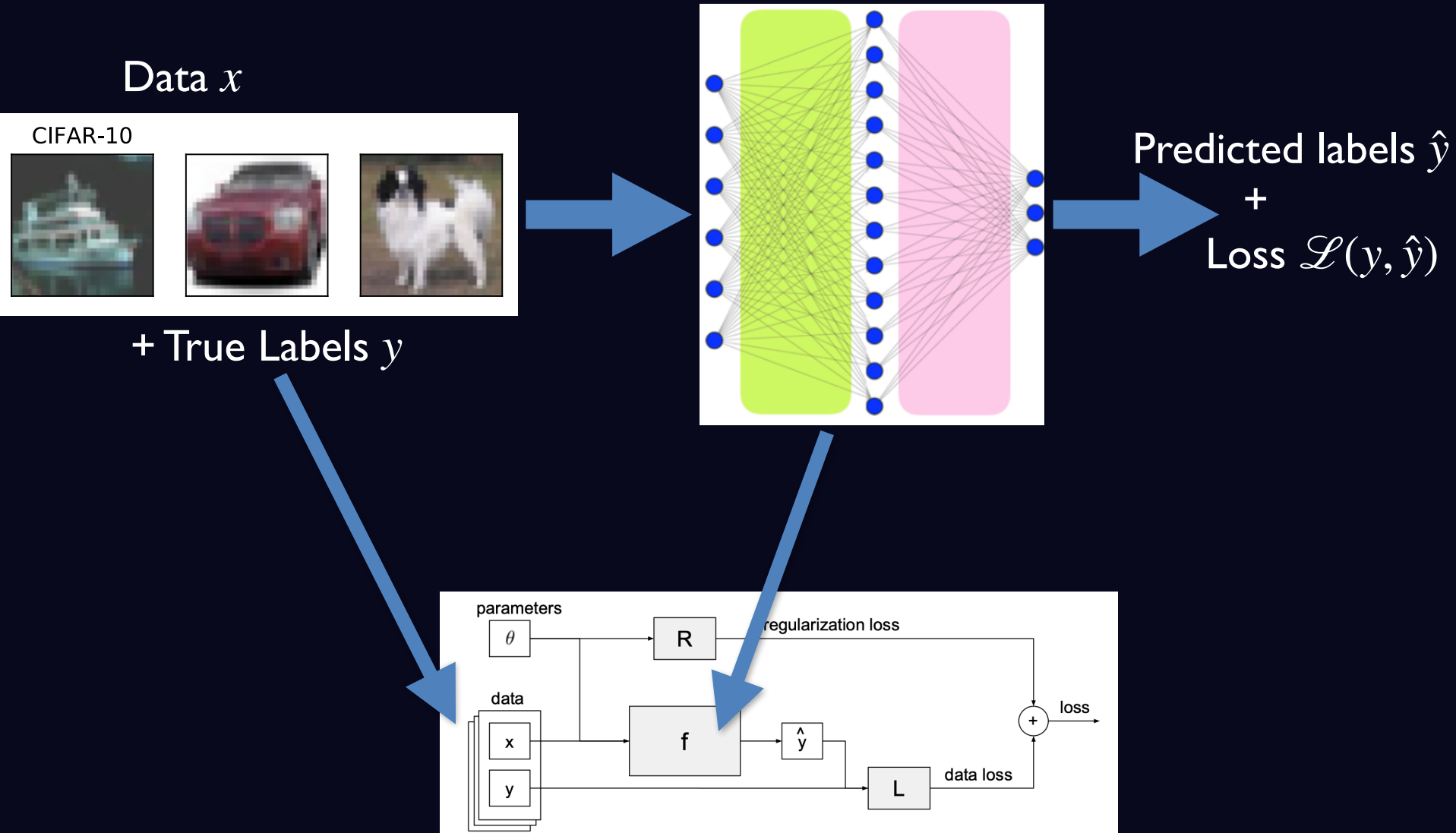
Lecture 1: Intro to Neural Networks

Dr Tiffany Vlaar
University of Glasgow

Framework

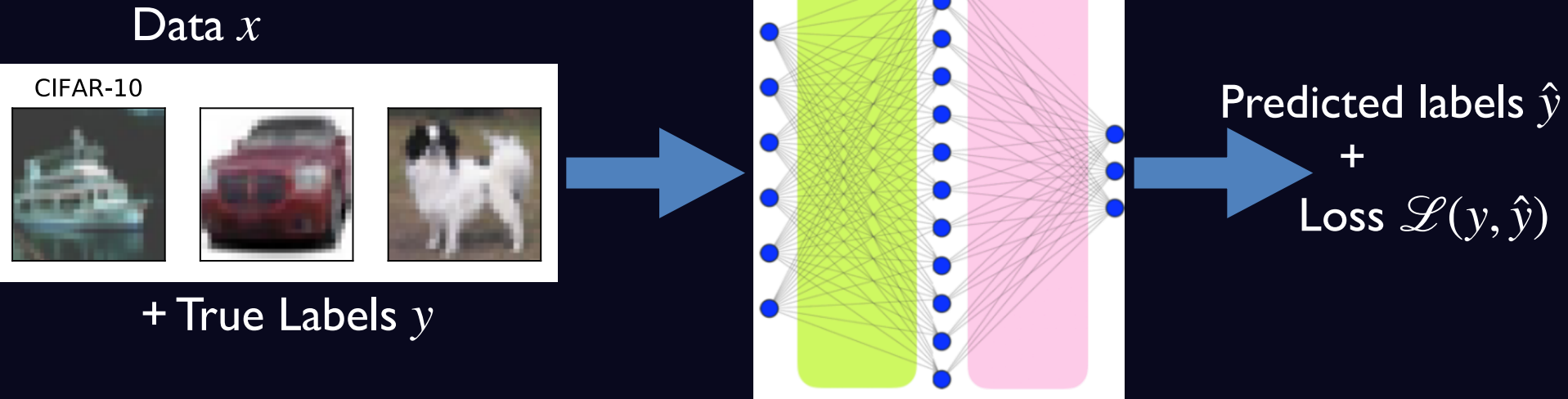


Framework

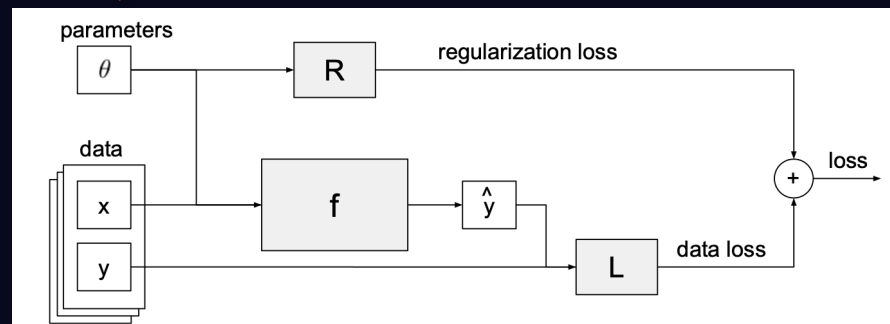


Karpathy, 2016

Framework

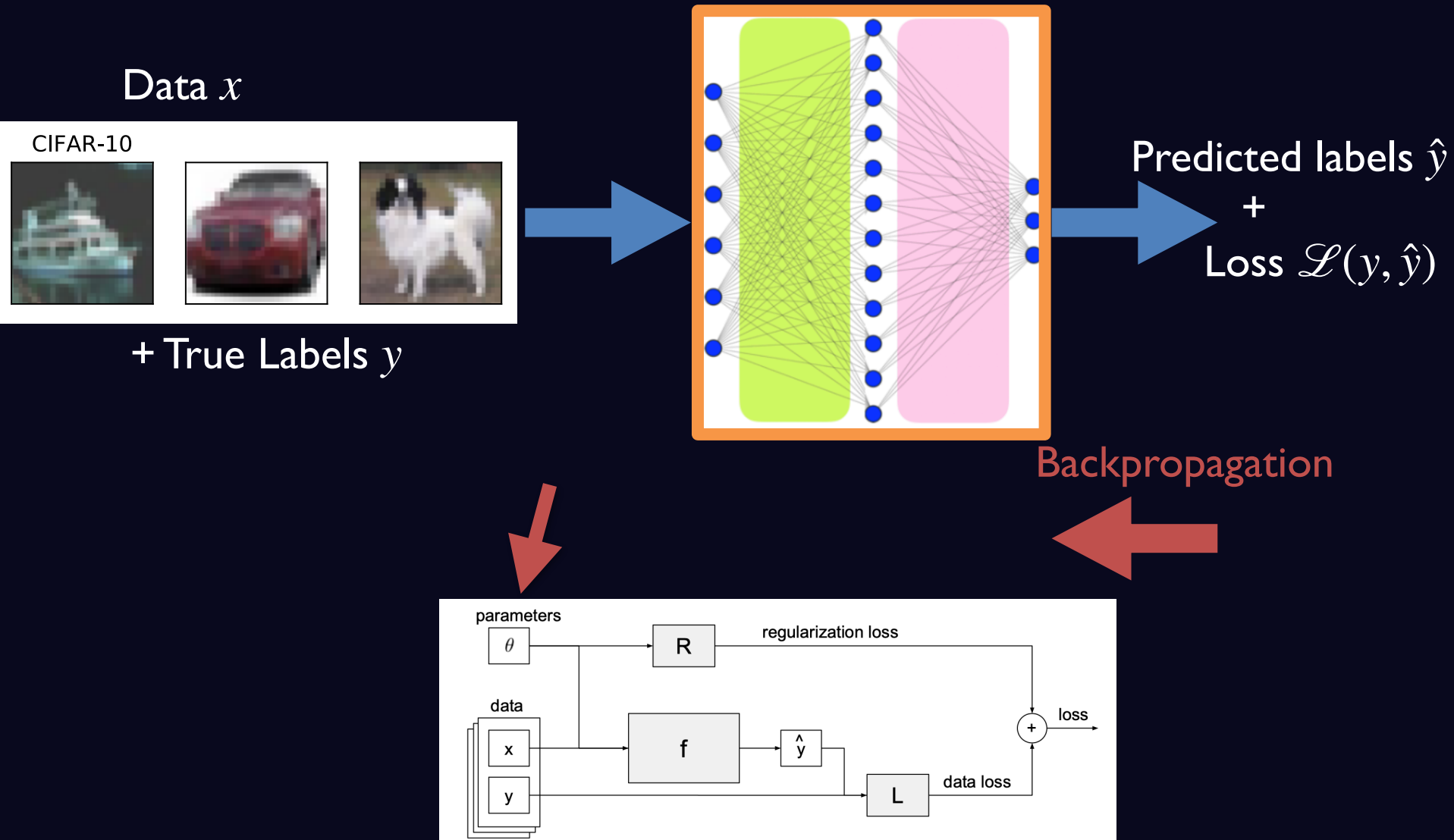


Backpropagation



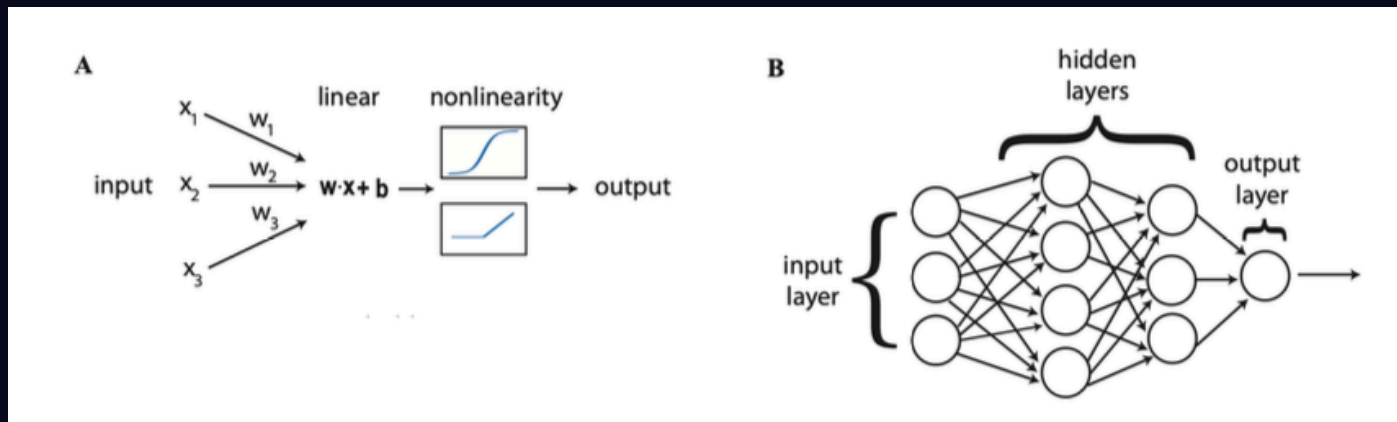
Karpathy, 2016

Framework



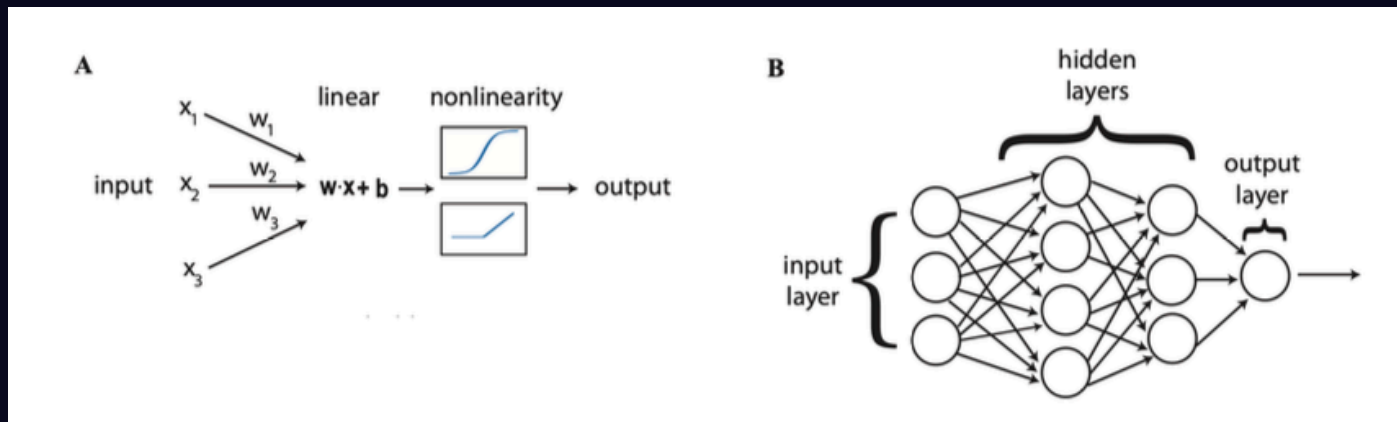
Karpathy, 2016

Multilayer Perceptron



Mehta et al., 2018

Multilayer Perceptron



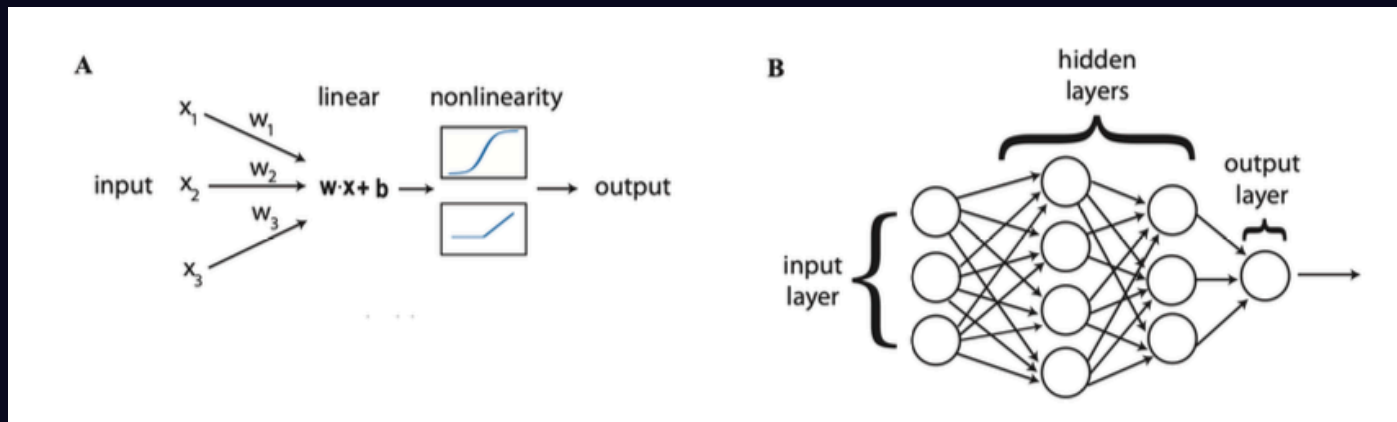
Mehta et al., 2018

If one hidden layer, with input data $x \in \mathbb{R}^N$, and no activation for output layer:

$$\hat{y} = \sigma(W_1 x + b_1) W_2 + b_2$$

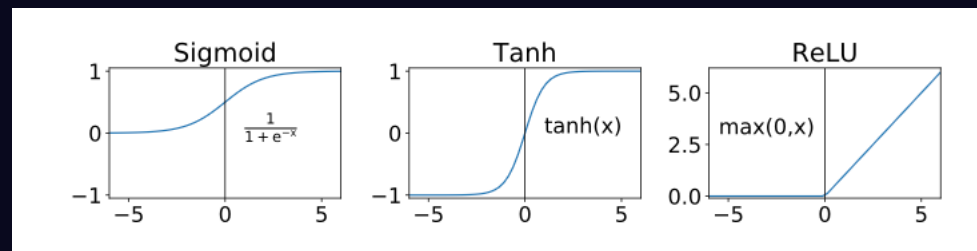
with weight matrices $W_1 \in \mathbb{R}^{N \times H}$, $W_2 \in \mathbb{R}^{H \times Z}$, bias vectors $b_1 \in \mathbb{R}^H$, $b_2 \in \mathbb{R}^Z$, and element-wise non-linearity σ

Multilayer Perceptron

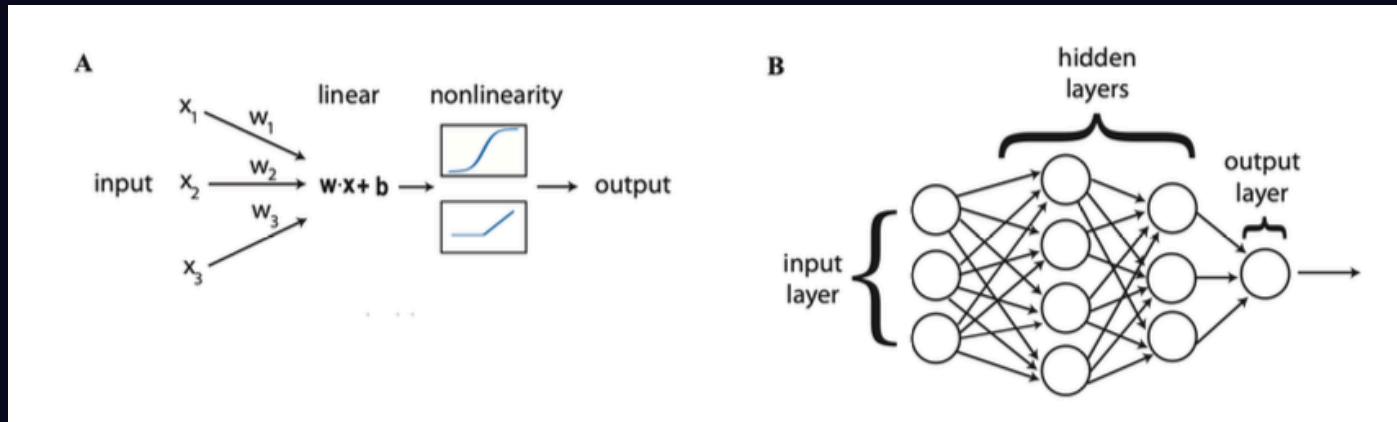


Mehta et al., 2018

Some choices for non-linear activation functions



Multilayer Perceptron

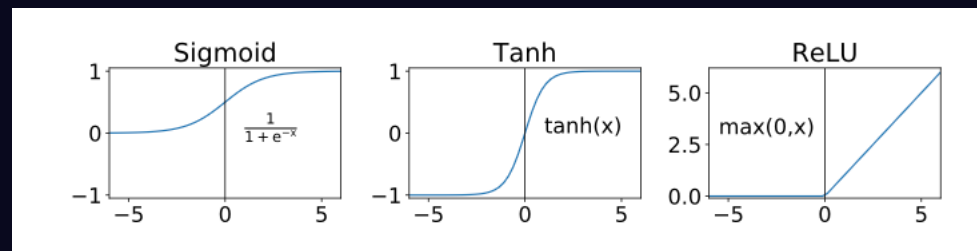


Mehta et al., 2018

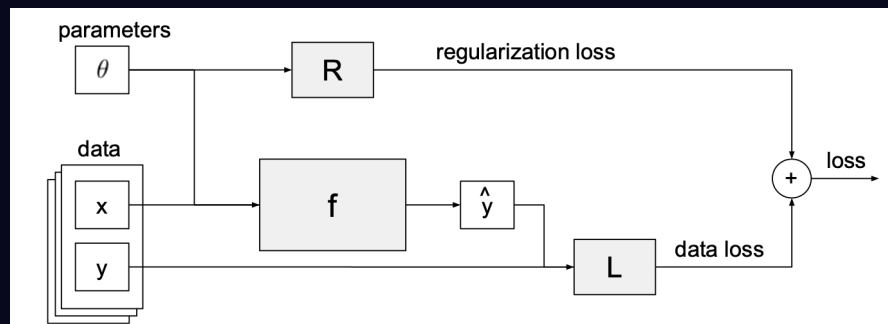
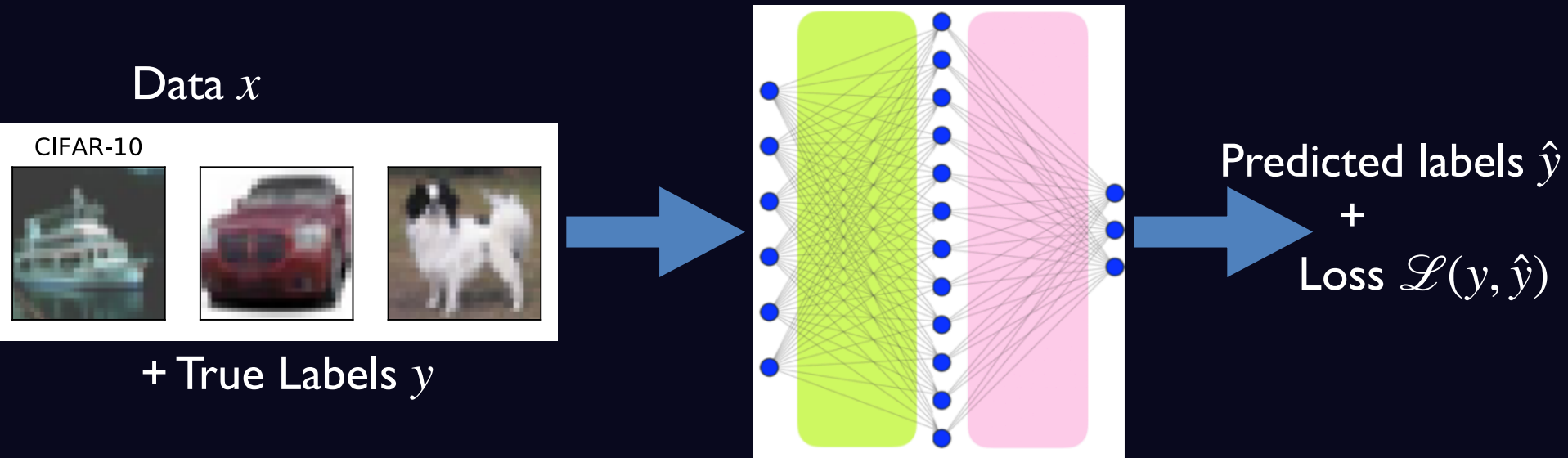
Depth: Number of layers

Width: Number of neurons in each layer

Some choices for non-linear activation functions

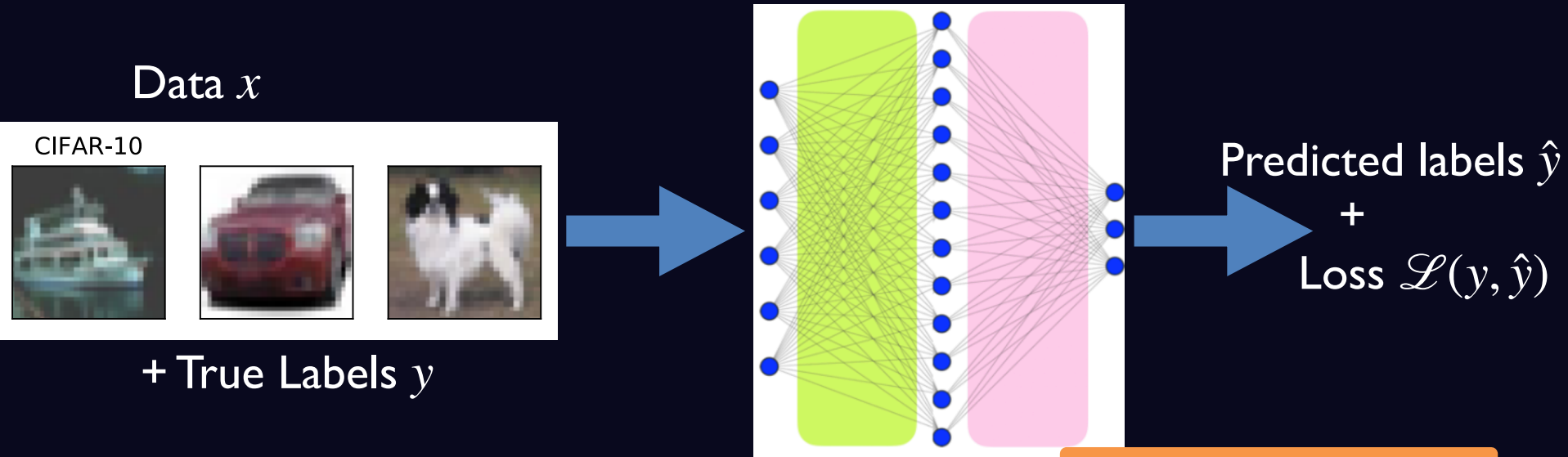


Framework

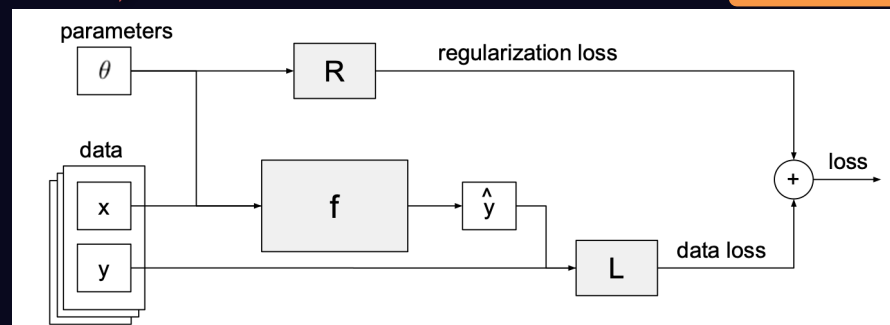


Karpathy, 2016

Framework



Backpropagation



Karpathy, 2016

Backpropagation

D.E. Rumelhart, G. E. Hinton, and R. J. Williams. Learning internal representations by error propagation. *Parallel Distributed Processing: Explorations in the Microstructure of Cognition*, page 318–362, 1986.

D. E. Rumelhart, G. E. Hinton, and R. J. Williams. Learning representations by backpropagating errors. *Nature*, 323(6088):533–536, 1986.

Backpropagation Example

Consider a feedforward neural network with one hidden layer, tanh activation, and no activation or bias for the output layer

Backpropagation Example

Consider a feedforward neural network with one hidden layer, tanh activation, and no activation or bias for the output layer

$$\begin{aligned} \text{pass input } x \text{ through first layer: } y' &= \tanh(W_1x + B_1), \\ \hat{y} &= W_2y'. \end{aligned}$$

Backpropagation Example

Consider a feedforward neural network with one hidden layer, tanh activation, and no activation or bias for the output layer

$$\begin{aligned} \text{pass input } x \text{ through first layer: } y' &= \tanh(W_1x + B_1), \\ \hat{y} &= W_2y'. \end{aligned}$$

Gradients of the loss with respect to the parameters:

Backpropagation Example

Consider a feedforward neural network with one hidden layer, tanh activation, and no activation or bias for the output layer

$$\begin{aligned} \text{pass input } x \text{ through first layer: } y' &= \tanh(W_1 x + B_1), \\ \hat{y} &= W_2 y'. \end{aligned}$$

Gradients of the loss with respect to the parameters:

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial W_2} &= \frac{\partial \mathcal{L}(y, \hat{y})}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial W_2} = \frac{\partial \mathcal{L}(y, \hat{y})}{\partial \hat{y}} y', \\ \frac{\partial \mathcal{L}}{\partial W_1} &= \frac{\partial \mathcal{L}(y, \hat{y})}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial y'} \frac{\partial y'}{\partial W_1} \end{aligned}$$

Backpropagation Example

Consider a feedforward neural network with one hidden layer, tanh activation, and no activation or bias for the output layer

$$\begin{aligned} \text{pass input } x \text{ through first layer: } y' &= \tanh(W_1 x + B_1), \\ \hat{y} &= W_2 y'. \end{aligned}$$

Gradients of the loss with respect to the parameters:

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial W_2} &= \frac{\partial \mathcal{L}(y, \hat{y})}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial W_2} = \frac{\partial \mathcal{L}(y, \hat{y})}{\partial \hat{y}} y', \\ \frac{\partial \mathcal{L}}{\partial W_1} &= \frac{\partial \mathcal{L}(y, \hat{y})}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial y'} \frac{\partial y'}{\partial W_1} = \frac{\partial \mathcal{L}(y, \hat{y})}{\partial \hat{y}} W_2 \frac{x}{\cosh^2(W_1 x + B_1)}, \end{aligned}$$

Backpropagation Example

Consider a feedforward neural network with one hidden layer, tanh activation, and no activation or bias for the output layer

$$\begin{aligned}\text{pass input } x \text{ through first layer: } y' &= \tanh(W_1 x + B_1), \\ \hat{y} &= W_2 y' .\end{aligned}$$

Gradients of the loss with respect to the parameters:

$$\begin{aligned}\frac{\partial \mathcal{L}}{\partial W_2} &= \frac{\partial \mathcal{L}(y, \hat{y})}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial W_2} = \frac{\partial \mathcal{L}(y, \hat{y})}{\partial \hat{y}} y', \\ \frac{\partial \mathcal{L}}{\partial W_1} &= \frac{\partial \mathcal{L}(y, \hat{y})}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial y'} \frac{\partial y'}{\partial W_1} = \frac{\partial \mathcal{L}(y, \hat{y})}{\partial \hat{y}} W_2 \frac{x}{\cosh^2(W_1 x + B_1)}, \\ \frac{\partial \mathcal{L}}{\partial B_1} &= \frac{\partial \mathcal{L}(y, \hat{y})}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial y'} \frac{\partial y'}{\partial B_1} = \frac{\partial \mathcal{L}(y, \hat{y})}{\partial \hat{y}} W_2 \frac{1}{\cosh^2(W_1 x + B_1)},\end{aligned}$$

Backpropagation Example

Consider a feedforward neural network with one hidden layer, tanh activation, and no activation or bias for the output layer

$$\begin{aligned}\text{pass input } x \text{ through first layer: } y' &= \tanh(W_1x + B_1), \\ \hat{y} &= W_2y'.\end{aligned}$$

Gradients of the loss with respect to the parameters:

$$\begin{aligned}\frac{\partial \mathcal{L}}{\partial W_2} &= \frac{\partial \mathcal{L}(y, \hat{y})}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial W_2} = \frac{\partial \mathcal{L}(y, \hat{y})}{\partial \hat{y}} y', \\ \frac{\partial \mathcal{L}}{\partial W_1} &= \frac{\partial \mathcal{L}(y, \hat{y})}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial y'} \frac{\partial y'}{\partial W_1} = \frac{\partial \mathcal{L}(y, \hat{y})}{\partial \hat{y}} W_2 \frac{x}{\cosh^2(W_1x + B_1)}, \\ \frac{\partial \mathcal{L}}{\partial B_1} &= \frac{\partial \mathcal{L}(y, \hat{y})}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial y'} \frac{\partial y'}{\partial B_1} = \frac{\partial \mathcal{L}(y, \hat{y})}{\partial \hat{y}} W_2 \frac{1}{\cosh^2(W_1x + B_1)},\end{aligned}$$

for a MSE (mean square error) loss $\partial \mathcal{L}(y, \hat{y}) / \partial \hat{y}$ is simply $2(\hat{y} - y)$

Neural Network Training

Neural Network Training

Basis: $d\theta = G(\theta) dt$, with neural network parameters $\theta \in \mathbb{R}^n$
and G the negative gradient of the loss of entire dataset.

Neural Network Training

Basis: $d\theta = G(\theta) dt$, with neural network parameters $\theta \in \mathbb{R}^n$
and G the negative gradient of the loss of entire dataset.

Explicit Euler gives with step size h

$$\theta_{n+1} = \theta_n + hG(\theta_n).$$

Neural Network Training

Basis: $d\theta = G(\theta) dt$, with neural network parameters $\theta \in \mathbb{R}^n$
and G the negative gradient of the loss of entire dataset.

Explicit Euler gives with step size h

$$\theta_{n+1} = \theta_n + hG(\theta_n).$$

For sufficiently small h

$$\mathcal{L}(\theta_{n+1}) - \mathcal{L}(\theta_n) = \left(\mathcal{L}(\theta_n) + \frac{d\mathcal{L}}{d\theta_n}(\theta_{n+1} - \theta_n) + \dots \right) - \mathcal{L}(\theta_n) = -h \left(\frac{d\mathcal{L}}{d\theta_n} \right)^2 + \mathcal{O}(h^2)$$

Neural Network Training

Basis: $d\theta = G(\theta) dt$, with neural network parameters $\theta \in \mathbb{R}^n$
and G the negative gradient of the loss of entire dataset.

Explicit Euler gives with step size h

$$\theta_{n+1} = \theta_n + hG(\theta_n).$$

For sufficiently small h

$$\mathcal{L}(\theta_{n+1}) - \mathcal{L}(\theta_n) = \left(\mathcal{L}(\theta_n) + \frac{d\mathcal{L}}{d\theta_n}(\theta_{n+1} - \theta_n) + \dots \right) - \mathcal{L}(\theta_n) = -h \left(\frac{d\mathcal{L}}{d\theta_n} \right)^2 + \mathcal{O}(h^2)$$

Stochastic Gradient Descent (SGD):

train on randomly selected subset of training data

Neural Network Training

Basis: $d\theta = G(\theta) dt$, with neural network parameters $\theta \in \mathbb{R}^n$
and G the negative gradient of the loss of entire dataset.

Explicit Euler gives with step size h

$$\theta_{n+1} = \theta_n + hG(\theta_n).$$

For sufficiently small h

$$\mathcal{L}(\theta_{n+1}) - \mathcal{L}(\theta_n) = \left(\mathcal{L}(\theta_n) + \frac{d\mathcal{L}}{d\theta_n}(\theta_{n+1} - \theta_n) + \dots \right) - \mathcal{L}(\theta_n) = -h \left(\frac{d\mathcal{L}}{d\theta_n} \right)^2 + \mathcal{O}(h^2)$$

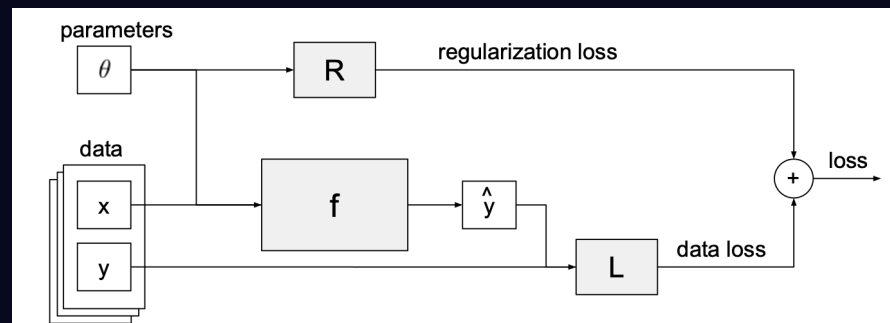
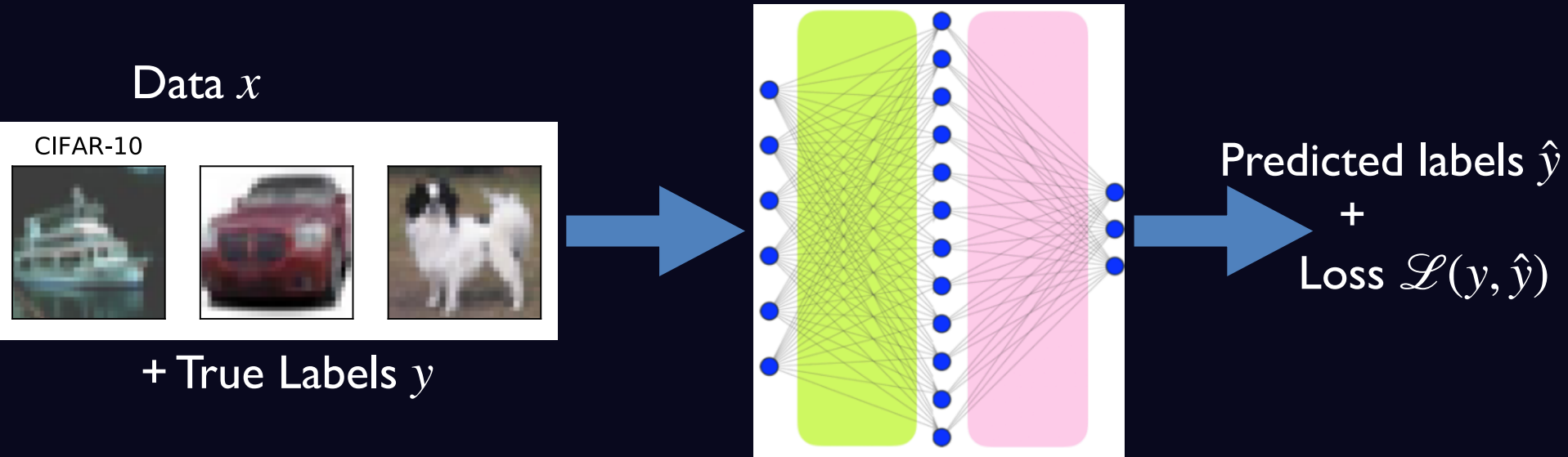
Stochastic Gradient Descent (SGD):

train on randomly selected subset of training data

$\Rightarrow$ replace pure gradient by noisy gradient $\tilde{G}(\theta)$.

Framework

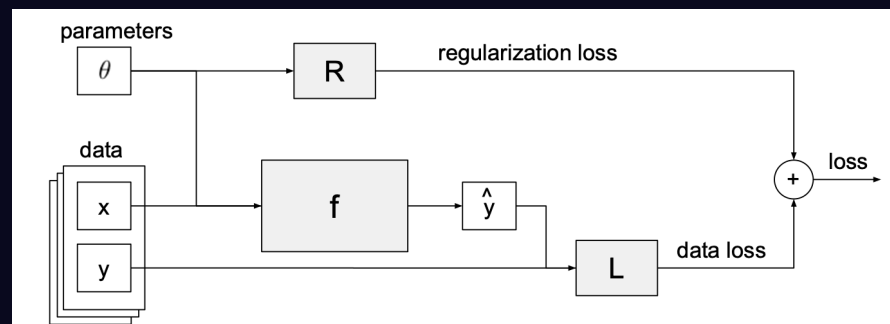
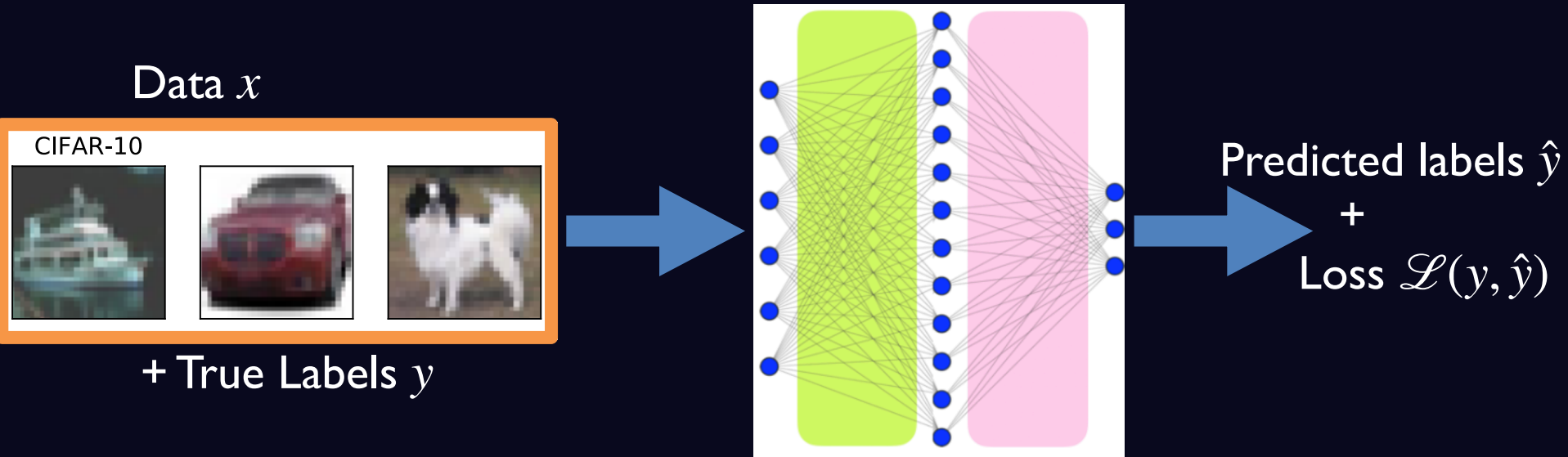
Classification Problems



Karpathy, 2016

Framework

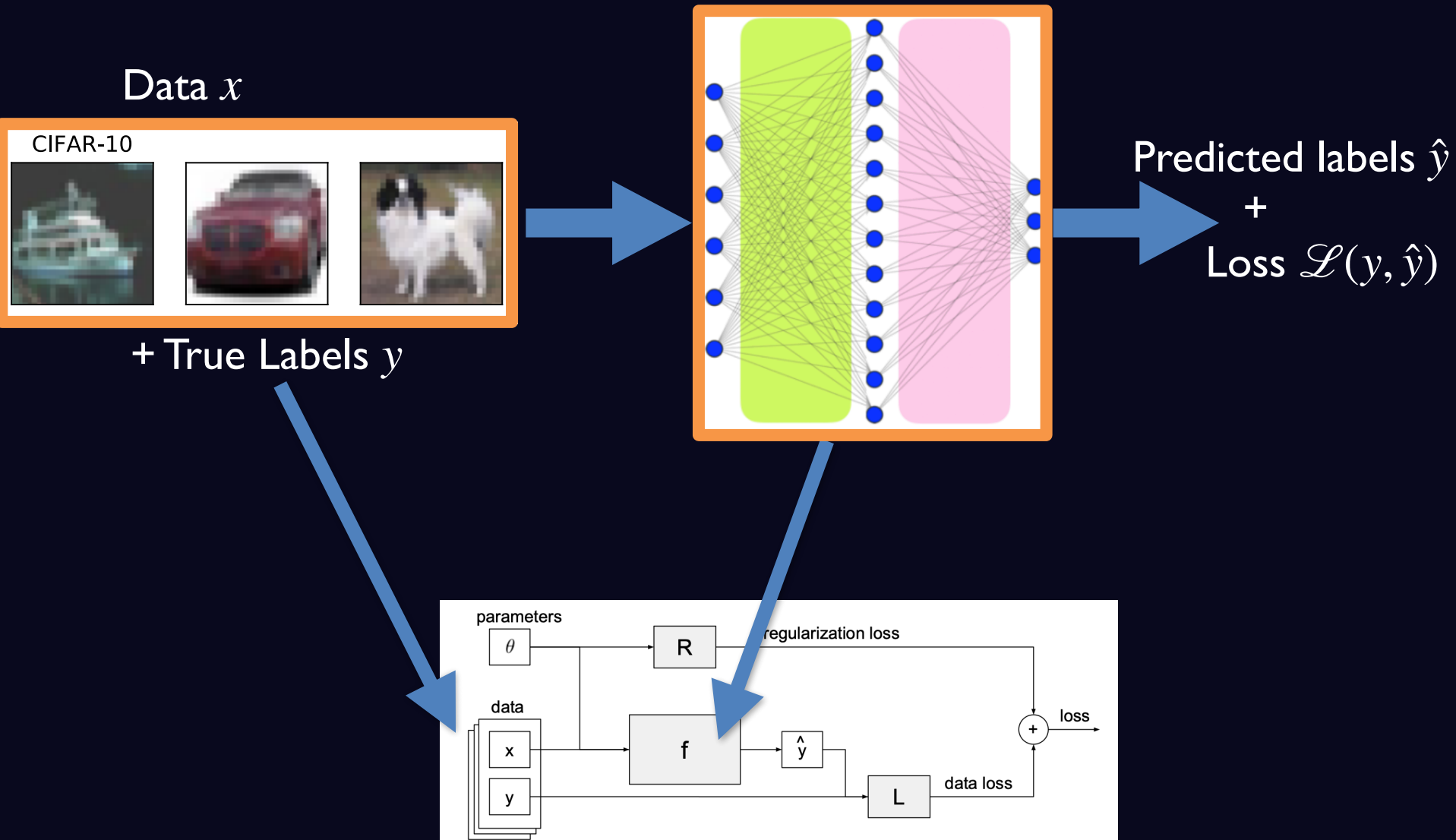
Classification Problems



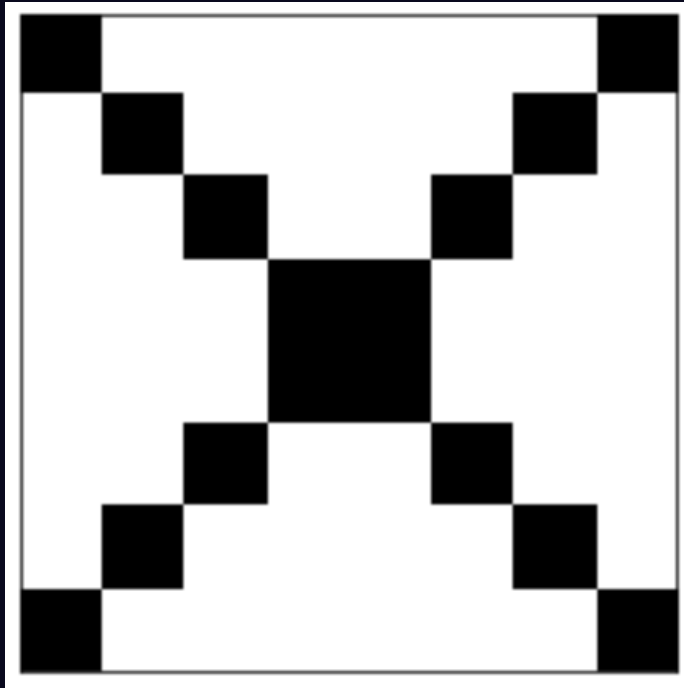
Karpathy, 2016

Framework

Classification Problems



Input: Images



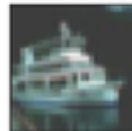
MNIST



Fashion-MNIST



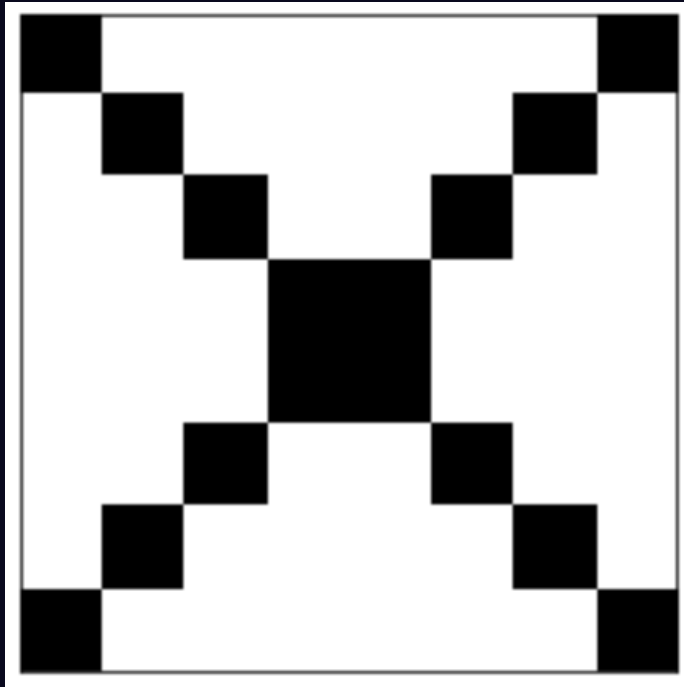
CIFAR-10



CIFAR-100

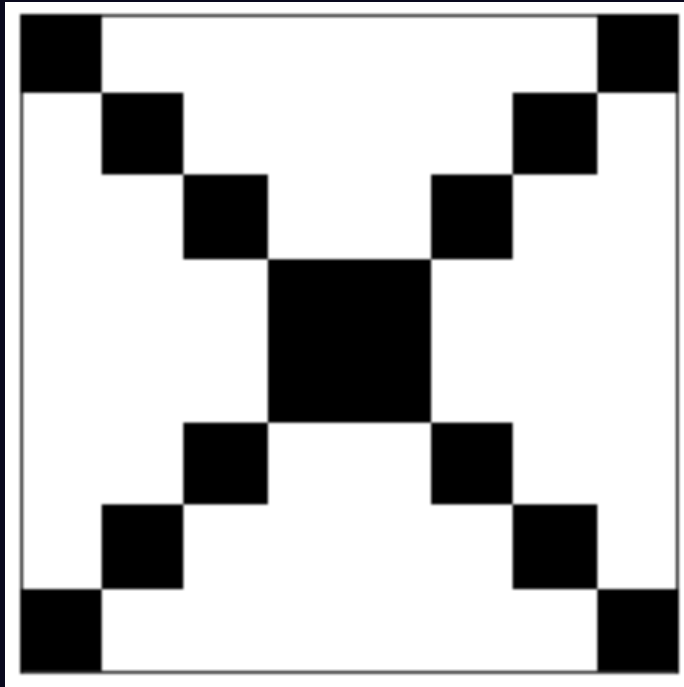


Input: Images

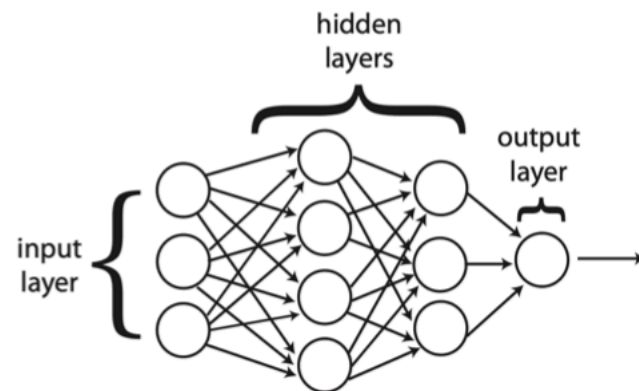
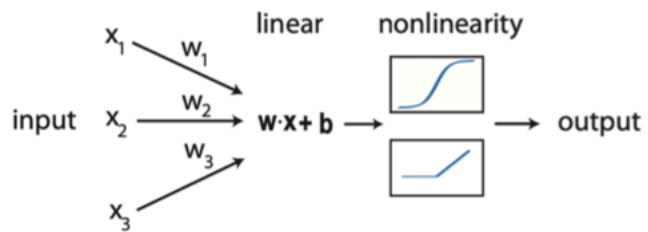


1	0	0	0	0	0	0	1
0	1	0	0	0	0	1	0
0	0	1	0	0	1	0	0
0	0	0	1	1	0	0	0
0	0	0	1	1	0	0	0
0	0	1	0	0	1	0	0
0	1	0	0	0	0	1	0
1	0	0	0	0	0	0	1

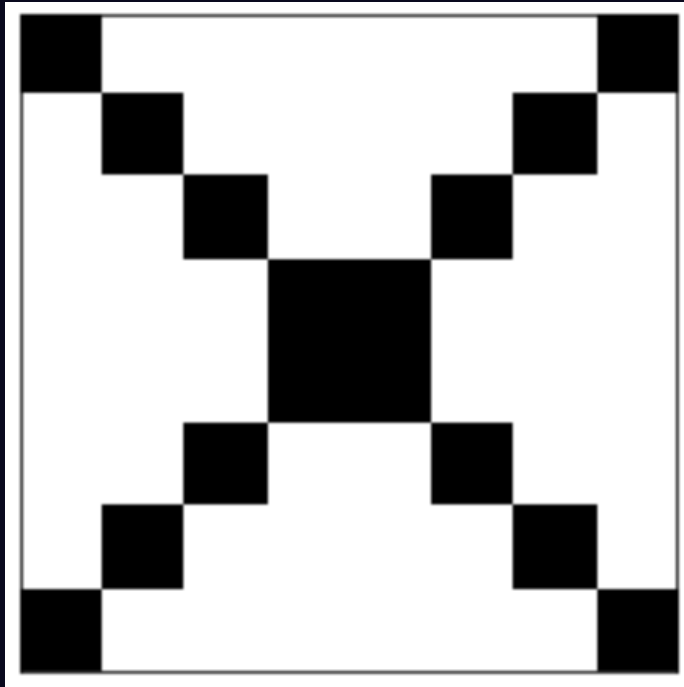
Input: Images



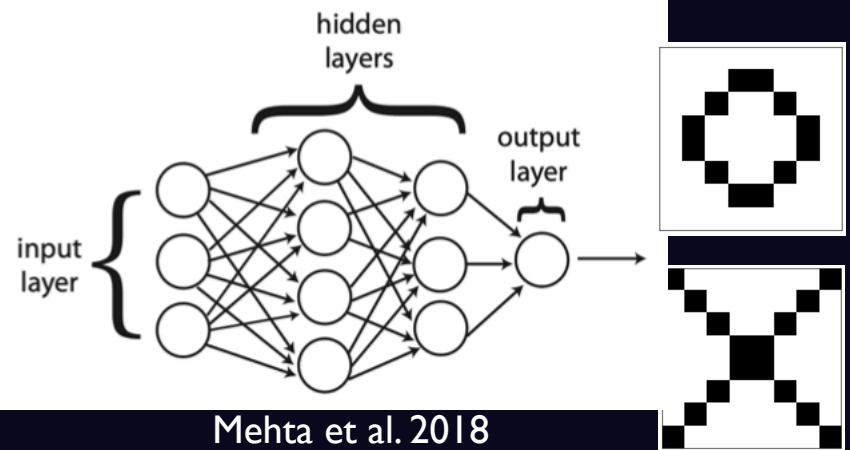
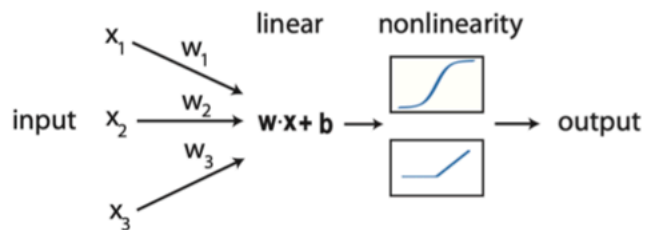
1	0	0	0	0	0	0	1
0	1	0	0	0	0	1	0
0	0	1	0	0	1	0	0
0	0	0	1	1	0	0	0
0	0	0	1	1	0	0	0
0	0	1	0	0	1	0	0
0	1	0	0	0	0	1	0
1	0	0	0	0	0	0	1



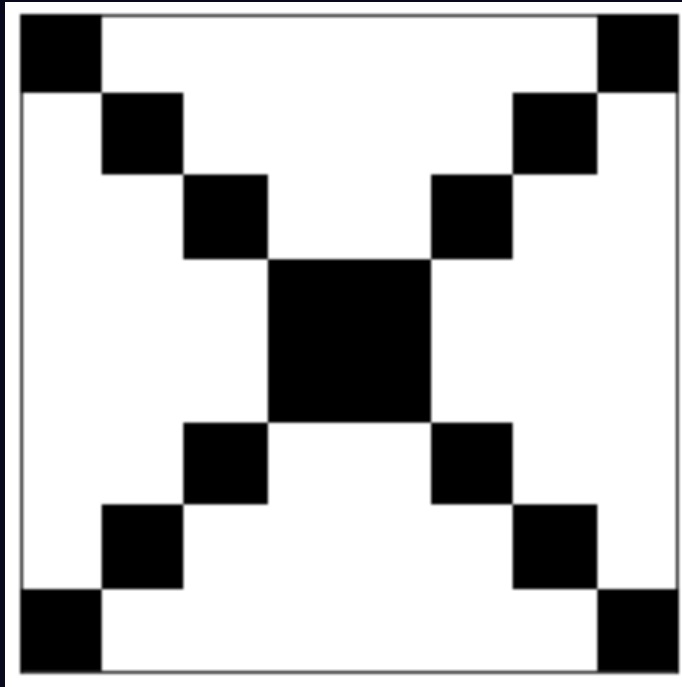
Input: Images



1	0	0	0	0	0	0	1
0	1	0	0	0	0	1	0
0	0	1	0	0	1	0	0
0	0	0	1	1	0	0	0
0	0	0	1	1	0	0	0
0	0	1	0	0	1	0	0
0	1	0	0	0	0	1	0
1	0	0	0	0	0	0	1

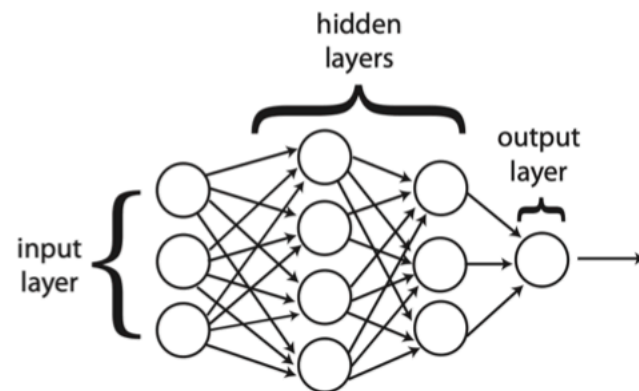
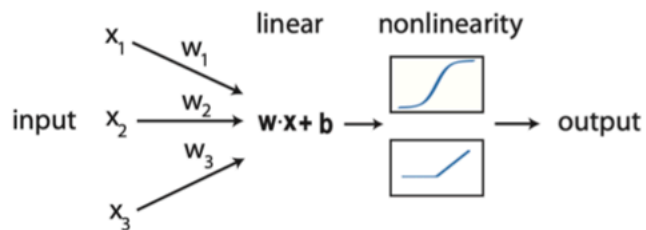


Input: Images

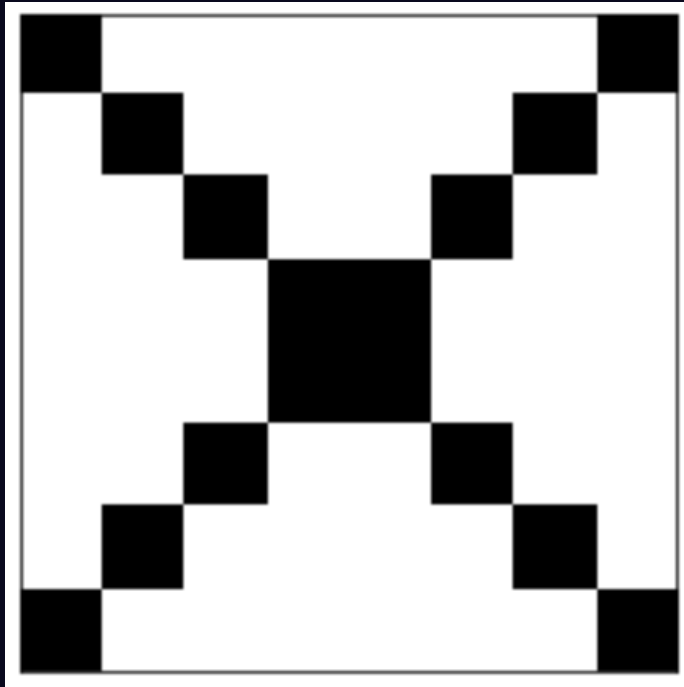


8x8 = 64 input nodes
for fully-connected

1	0	0	0	0	0	0	1
0	1	0	0	0	0	1	0
0	0	1	0	0	1	0	0
0	0	0	1	1	0	0	0
0	0	0	1	1	0	0	0
0	0	1	0	0	1	0	0
0	1	0	0	0	0	1	0
1	0	0	0	0	0	0	1



Input: Images



8x8 = 64 input nodes
for fully-connected

1	0	0	0	0	0	0	1
0	1	0	0	0	0	1	0
0	0	1	0	0	1	0	0
0	0	0	1	1	0	0	0
0	0	0	1	1	0	0	0
0	0	1	0	0	1	0	0
0	1	0	0	0	0	1	0
1	0	0	0	0	0	0	1

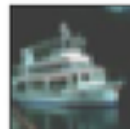
MNIST



Fashion-MNIST



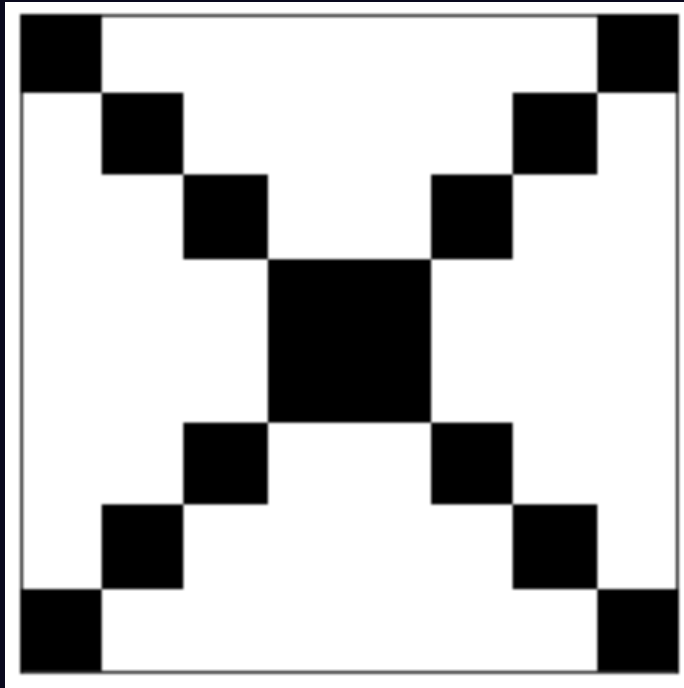
CIFAR-10



CIFAR-100



Input: Images



$8 \times 8 = 64$ input nodes
for fully-connected

1	0	0	0	0	0	0	1
0	1	0	0	0	0	1	0
0	0	1	0	0	1	0	0
0	0	0	1	1	0	0	0
0	0	0	1	1	0	0	0
0	0	1	0	0	1	0	0
0	1	0	0	0	0	1	0
1	0	0	0	0	0	0	1

$32 \times 32 \times 3 = 3072$ input nodes

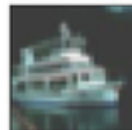
MNIST



Fashion-MNIST



CIFAR-10



CIFAR-100



Input: Images

1	0	0	0	0	0	0	1
0	1	0	0	0	0	1	0
0	0	1	0	0	1	0	0
0	0	0	1	1	0	0	0
0	0	0	1	1	0	0	0
0	0	1	0	0	1	0	0
0	1	0	0	0	0	1	0
1	0	0	0	0	0	0	1

Shifted, but still a cross

0	1	0	0	0	0	0	0
0	0	1	0	0	0	0	1
0	0	0	1	0	0	1	0
0	0	0	0	1	1	0	0
0	0	0	0	1	1	0	0
0	0	0	1	0	0	1	0
0	0	1	0	0	0	0	1
0	1	0	0	0	0	0	0

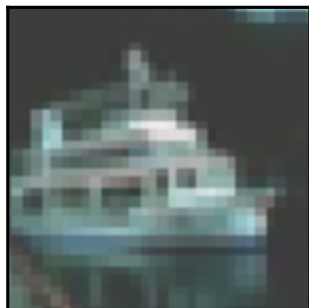
Input: Images

1	0	0	0	0	0	0	1
0	1	0	0	0	0	1	0
0	0	1	0	0	1	0	0
0	0	0	1	1	0	0	0
0	0	0	1	1	0	0	0
0	0	1	0	0	1	0	0
0	1	0	0	0	0	1	0
1	0	0	0	0	0	0	1

Shifted, but still a cross

0	1	0	0	0	0	0	0
0	0	1	0	0	0	0	1
0	0	0	1	0	0	1	0
0	0	0	0	1	1	0	0
0	0	0	0	1	1	0	0
0	0	0	1	0	0	1	0
0	0	1	0	0	0	0	1
0	1	0	0	0	0	0	0

CIFAR-10



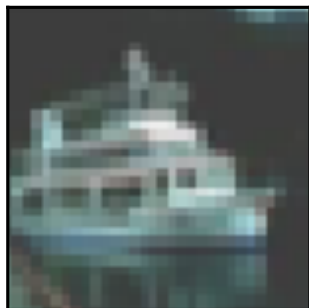
Input: Images

1	0	0	0	0	0	0	1
0	1	0	0	0	0	1	0
0	0	1	0	0	1	0	0
0	0	0	1	1	0	0	0
0	0	0	1	1	0	0	0
0	0	1	0	0	1	0	0
0	1	0	0	0	0	1	0
1	0	0	0	0	0	0	1

Shifted, but still a cross

0	1	0	0	0	0	0	0
0	0	1	0	0	0	0	1
0	0	0	1	0	0	1	0
0	0	0	0	1	1	0	0
0	0	0	0	1	1	0	0
0	0	0	1	0	0	1	0
0	0	1	0	0	0	0	1
0	1	0	0	0	0	0	0

CIFAR-10



Key ideas

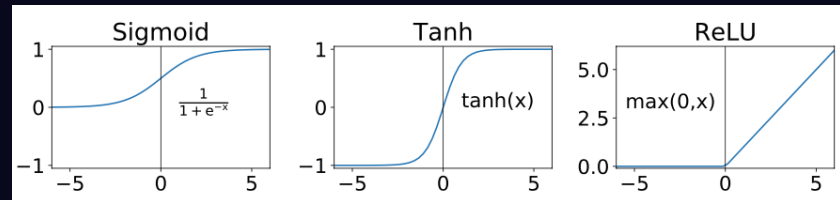
- Preserve spatial structure.
- Connect each neuron to only a local region of input volume.
- Parameter sharing.
- Translation equivariance.

Convolutional Neural Network (CNN) architecture components

- Convolution Layers
- Pooling Layers
- Fully Connected Layers
- Activation

Convolutional Neural Network (CNN) architecture components

- Convolution Layers
- Pooling Layers
- Fully Connected Layers
- Activation



Convolutional Neural Network (CNN) architecture components

- Convolution Layers
- Pooling Layers
- Fully Connected Layers
- Activation

Convolution Layer : **Set of learnable filters**

Image 32x32x3



Filter 4x4x3



Convolution Layer : Set of learnable filters

Image 32x32x3

3 (R, G, B) for color image
1 for greyscale

Filter 4x4x3



Convolution Layer : **Set of learnable filters**

Image 32x32x3



Filter 4x4x3



Filters are convolved with local regions as they slide across input volume

Filter 4x4x3



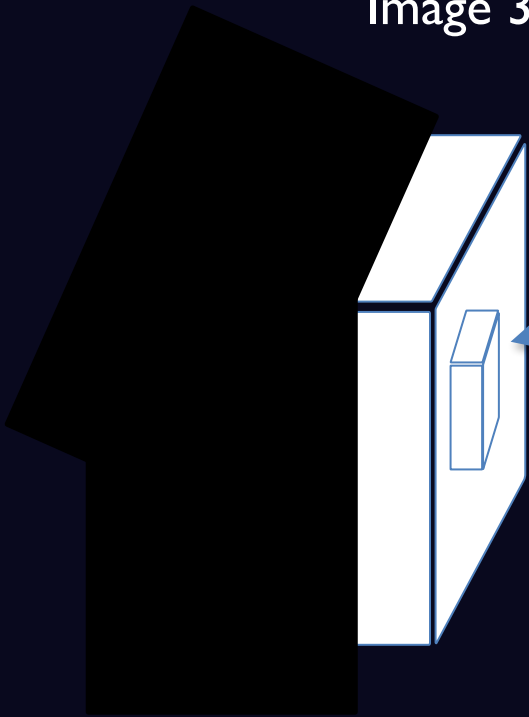
Convolution Layer

Image 32x32x3

Filters are convolved with local regions as they slide across input volume

Filter 4x4x3

Hyperparameters:
Filter size and number of filters



Convolution Layer

Image 32x32x3

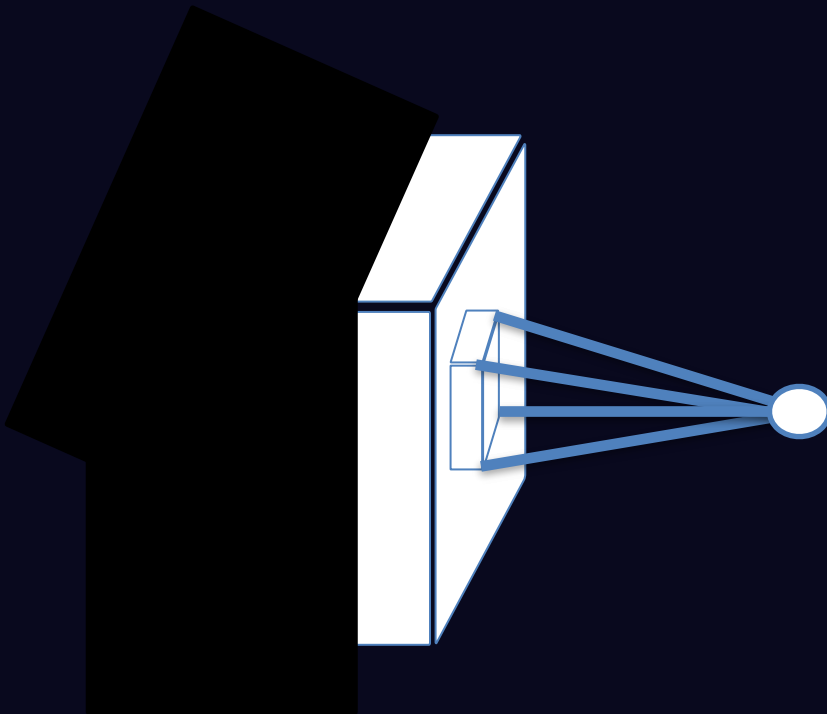
Filters are convolved with local regions as they slide across input volume

Filter 4x4x3

Hyperparameters:
Filter size and number of filters

Number of neurons looking at same region
= number of filters

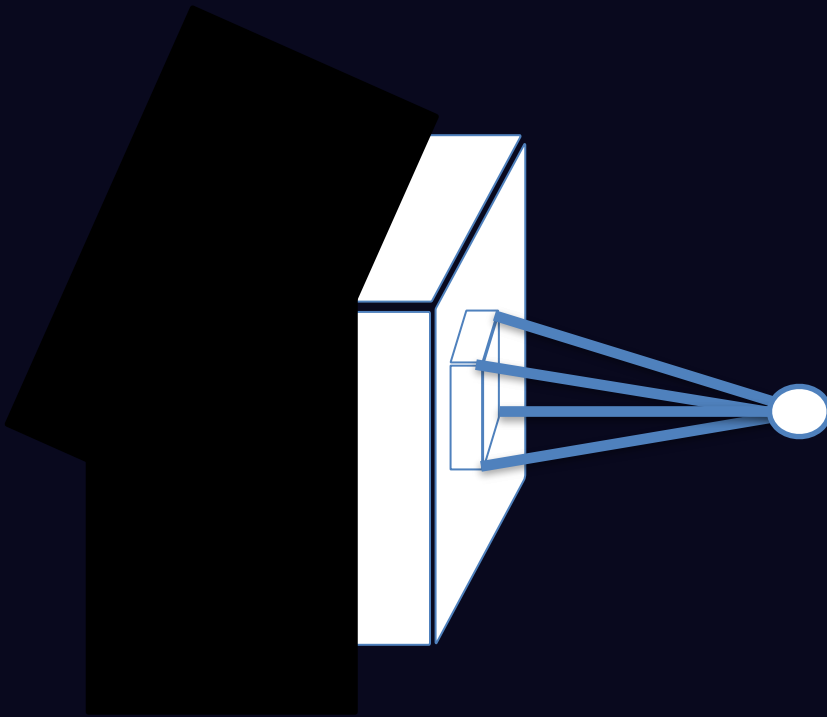




Convolve filter

Dot products between filter and the input

Convolution Layer

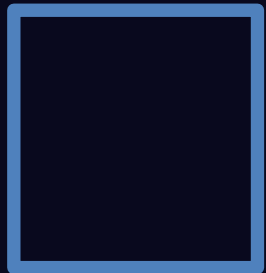


Convolve filter

Dot products between filter and the input

Convolution

Slide filter



over image

1	0	0	0	0	0	0	1
0	1	0	0	0	0	1	0
0	0	1	0	0	1	0	0
0	0	0	1	1	0	0	0
0	0	0	1	1	0	0	0
0	0	1	0	0	1	0	0
0	1	0	0	0	0	1	0
1	0	0	0	0	0	0	1

Convolution

Slide filter

a	b	c
d	e	f
g	h	i

over image

1	0	0	0	0	0	0	1
0	1	0	0	0	0	1	0
0	0	1	0	0	1	0	0
0	0	0	1	1	0	0	0
0	0	0	1	1	0	0	0
0	0	1	0	0	1	0	0
0	1	0	0	0	0	1	0
1	0	0	0	0	0	0	1

Convolution

Slide filter

a	b	c
d	e	f
g	h	i

over image

A	B	C				
D	E	F				
G	H	I				

Convolution

Slide filter

a	b	c
d	e	f
g	h	i

over image

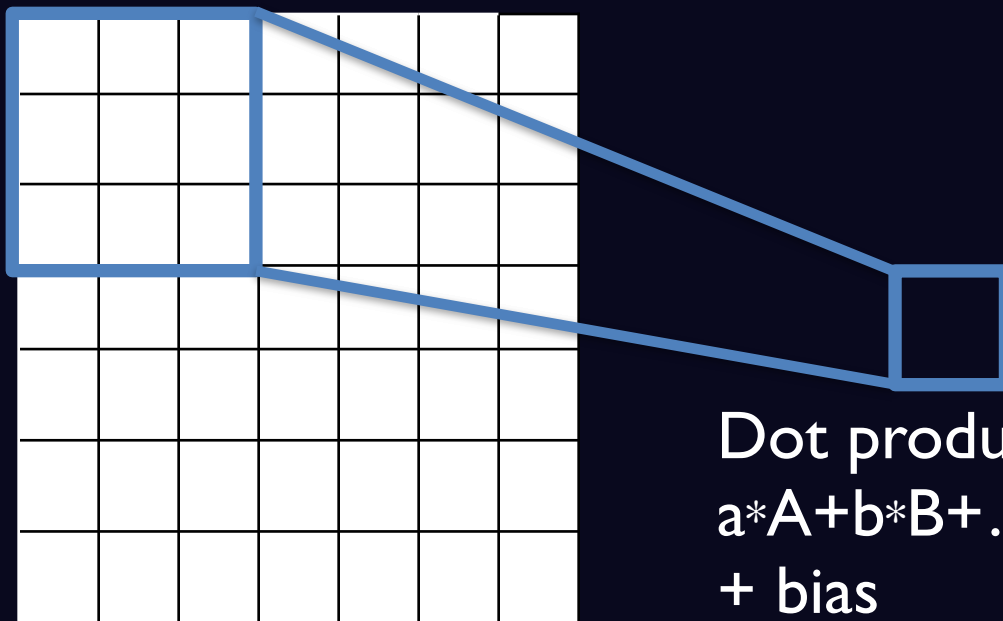
A	B	C				
D	E	F				
G	H	I				

Convolution

a	b	c
d	e	f
g	h	i

A	B	C				
D	E	F				
G	H	I				

Slide filter over image



Dot product
 $a*A + b*B + \dots + h*H + i*I$
+ bias

Data: A, B, ... , I

Through training improve: a, b, ... , i and bias

Convolution

a	b	c
d	e	f
g	h	i

Slide filter over image

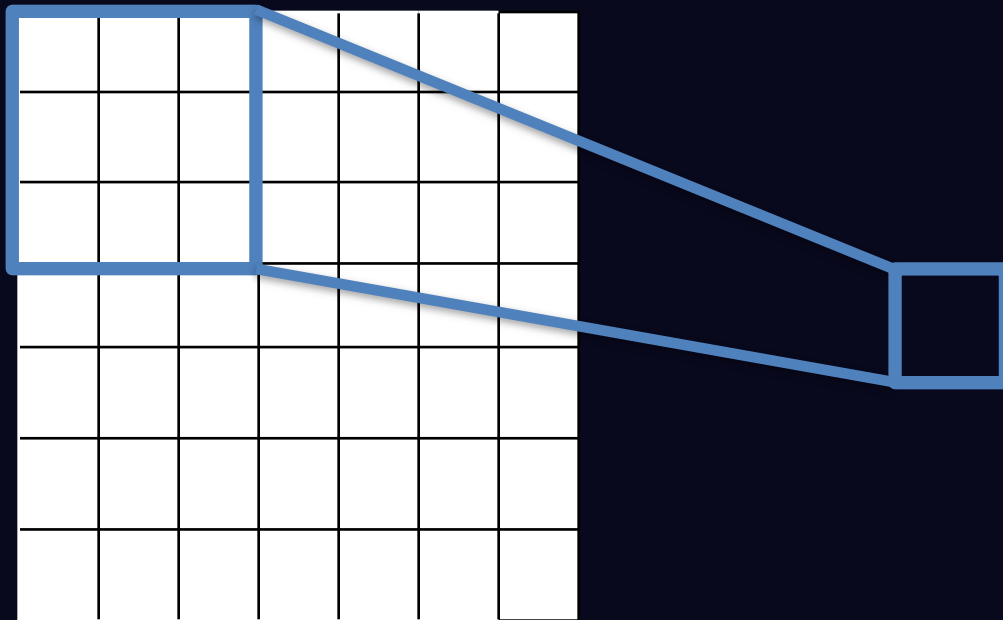
1	0	0				
0	1	0				
0	0	1				



Dot product: $a + e + i$

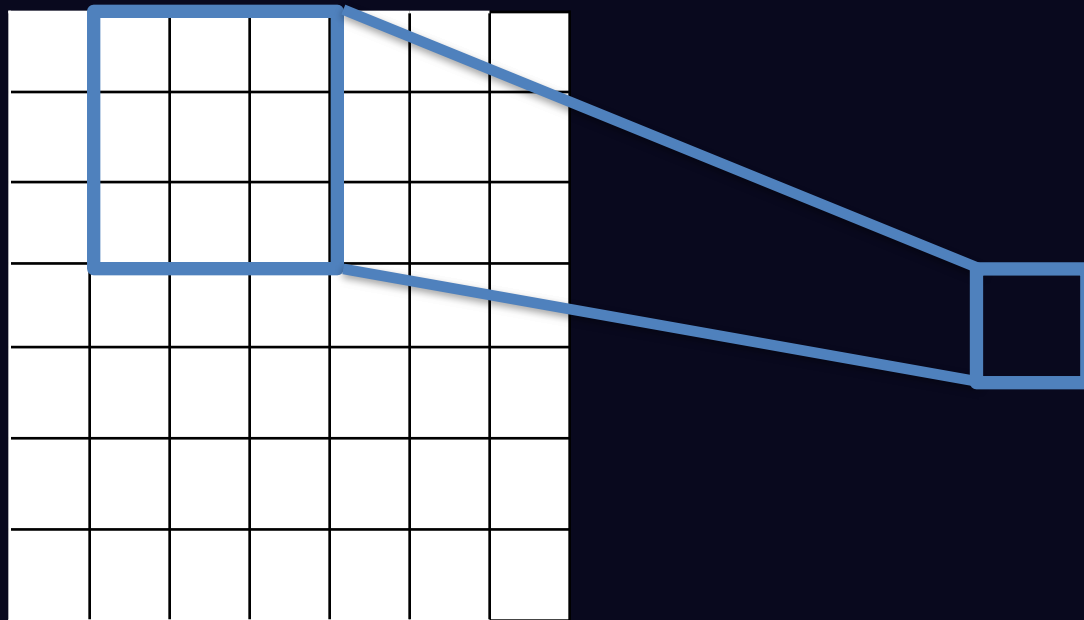
Convolution

Slide filter over image



Convolution

Slide filter over image

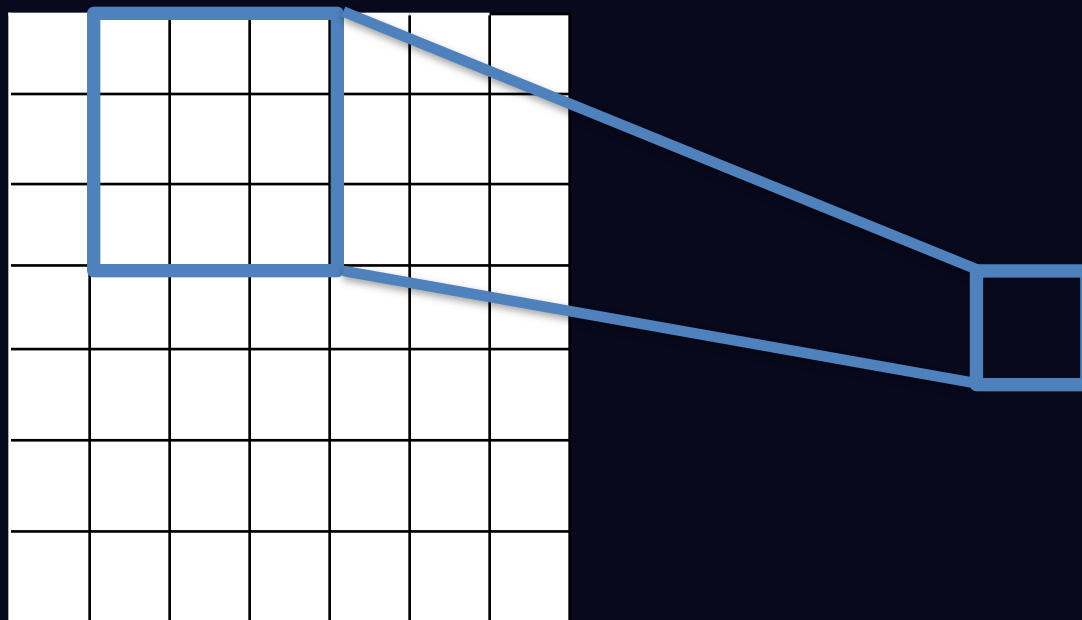


Convolution

a	b	c
d	e	f
g	h	i

Slide filter over image

	B	C				
	E	F				
	H	I				



Convolution

a	b	c
d	e	f
g	h	i

Slide filter over image

	0	0	0			
	1	0	0			
	0	1	0			



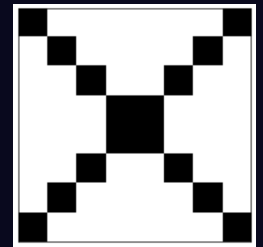
Dot product?

Convolution

a	b	c
d	e	f
g	h	i

Slide filter over image

	0	0	0			
	1	0	0			
	0	1	0			

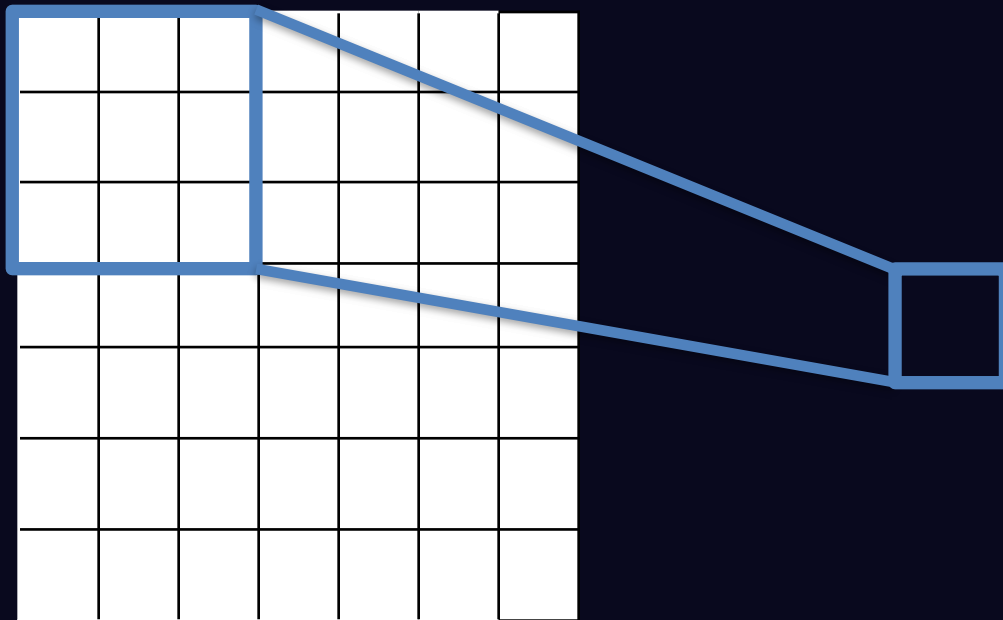


Dot product?

$$d + h$$

Convolution

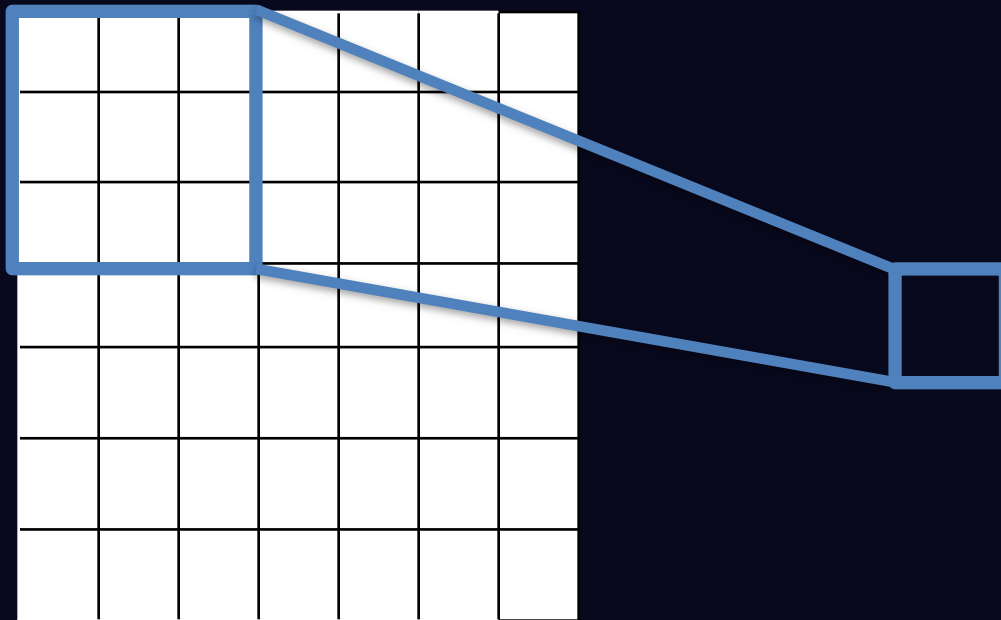
Slide filter over image



Convolution

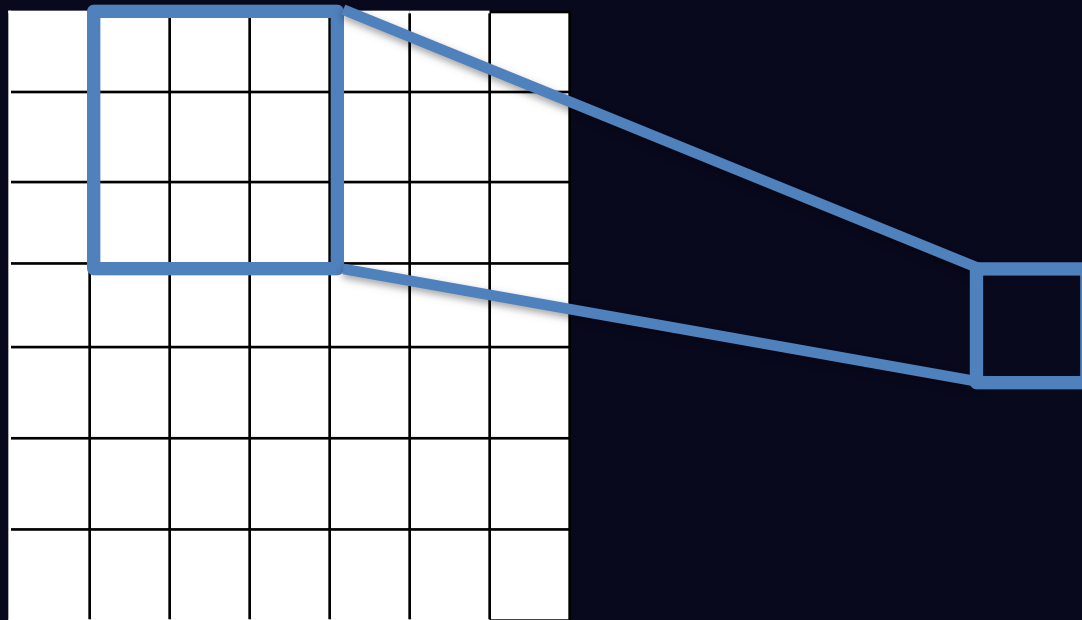
Slide filter over image

Stride 1



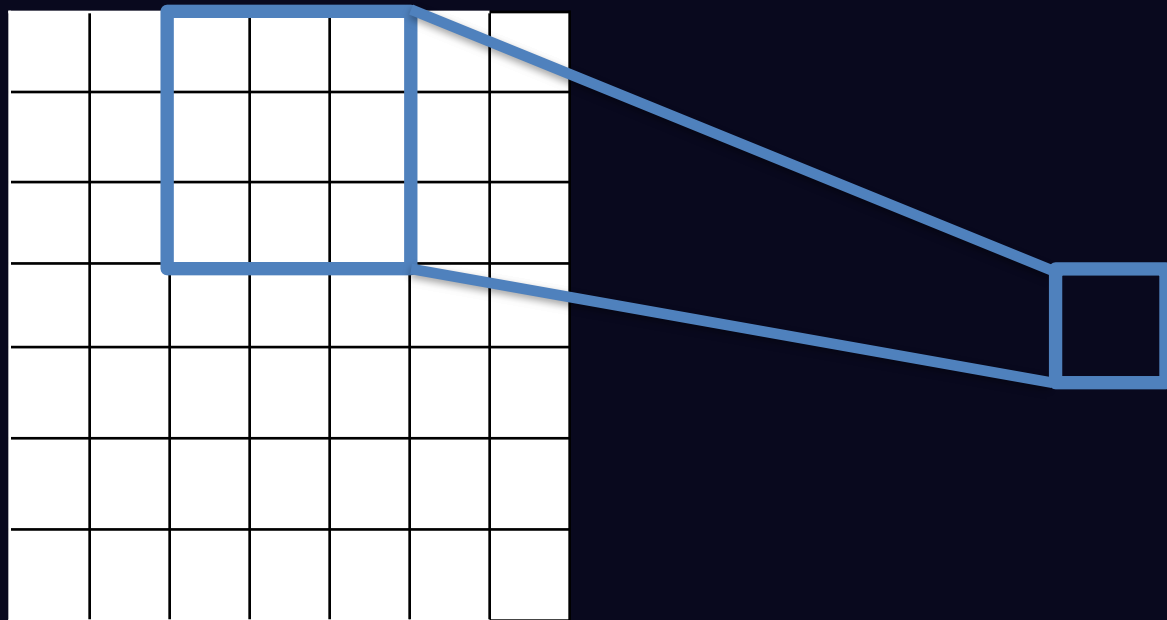
Convolution

Slide filter over image



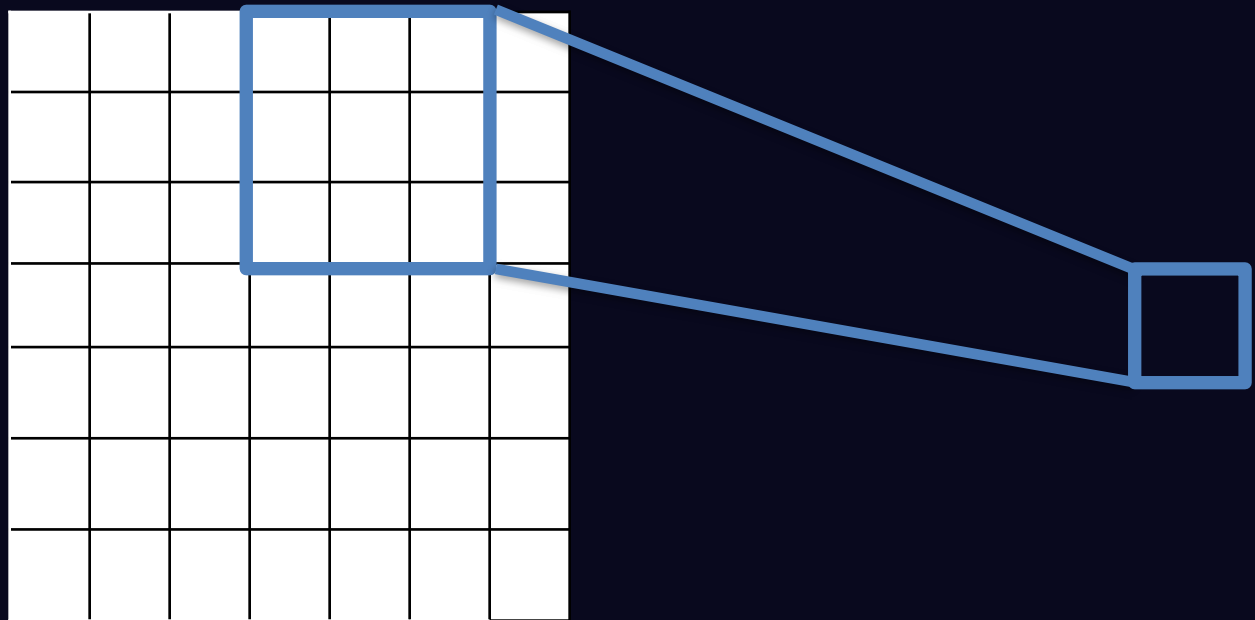
Convolution

Slide filter over image



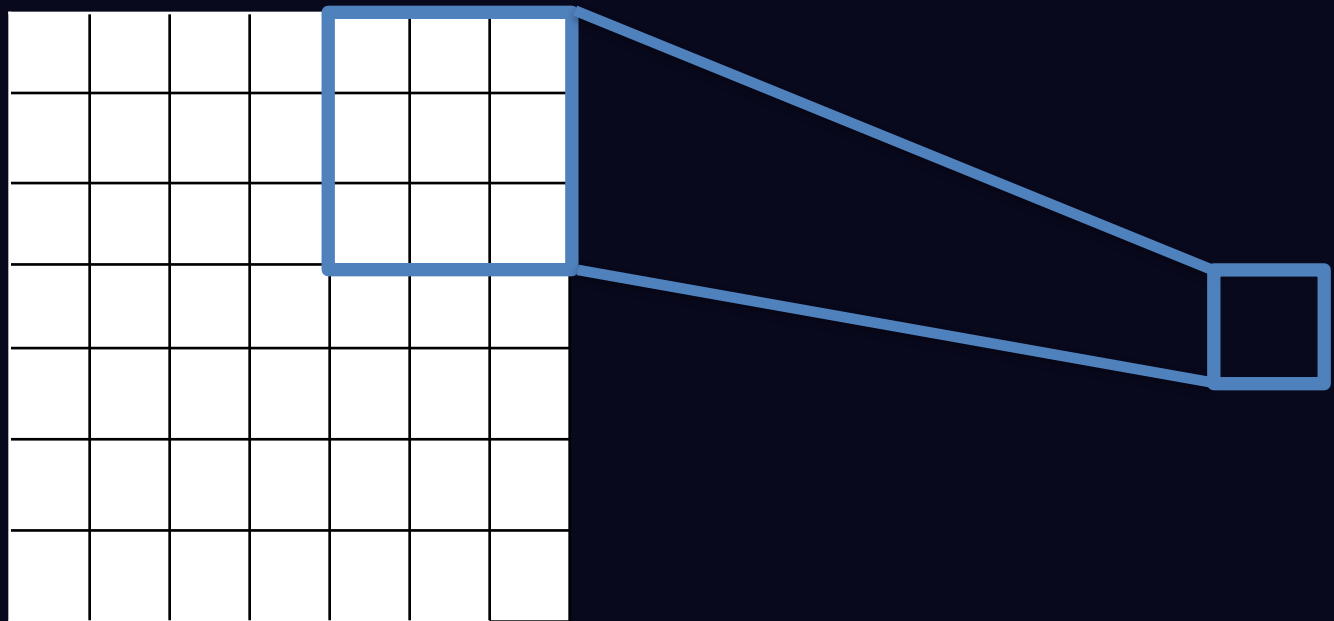
Convolution

Slide filter over image



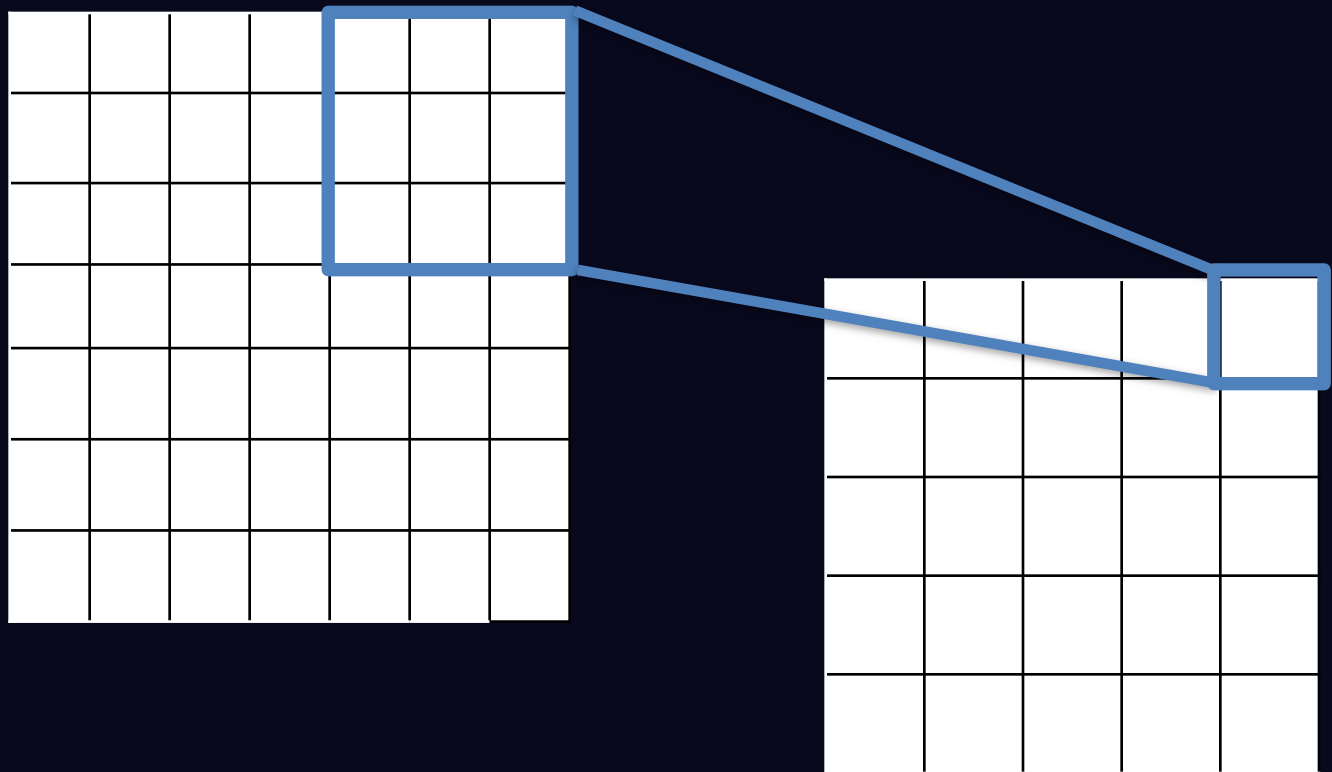
Convolution

Slide filter over image



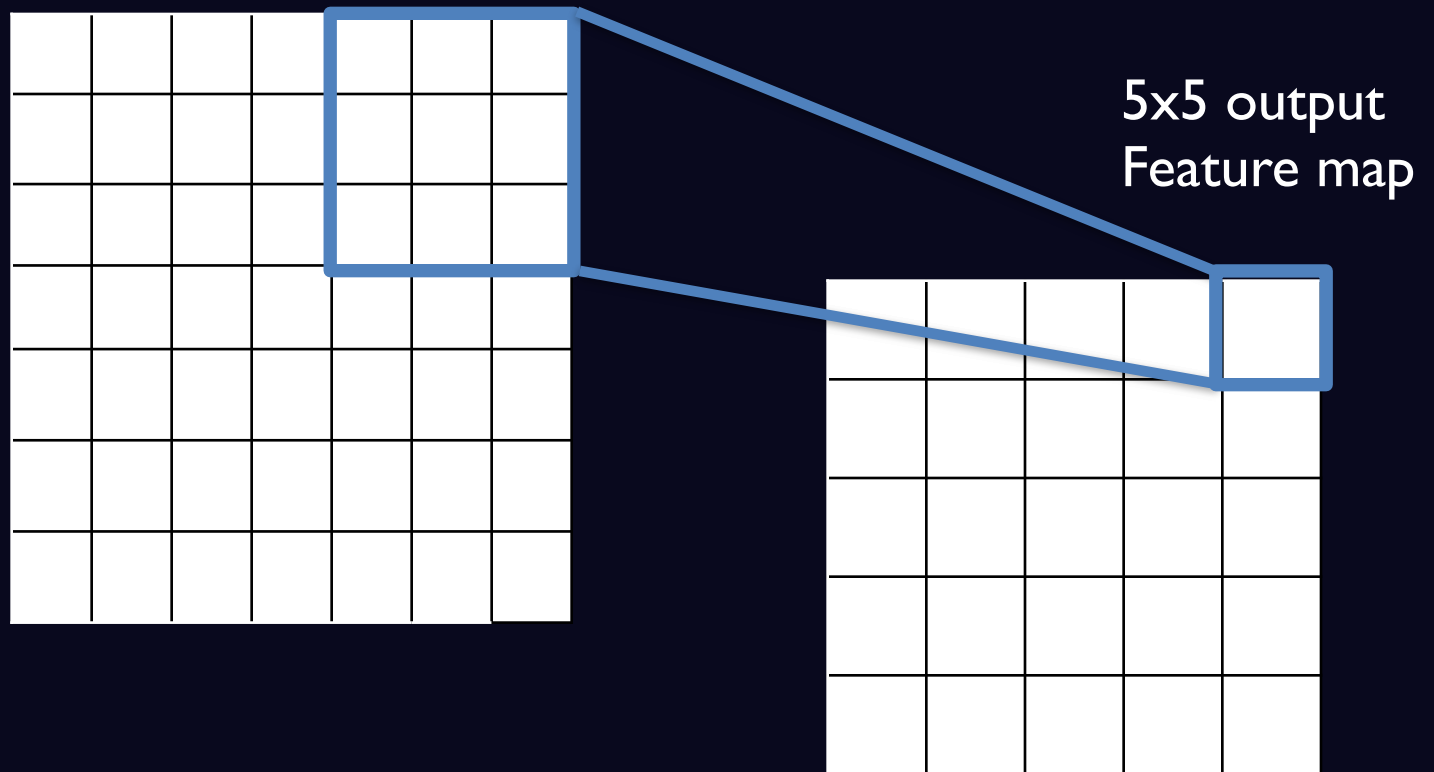
Convolution

Slide filter over image



Convolution

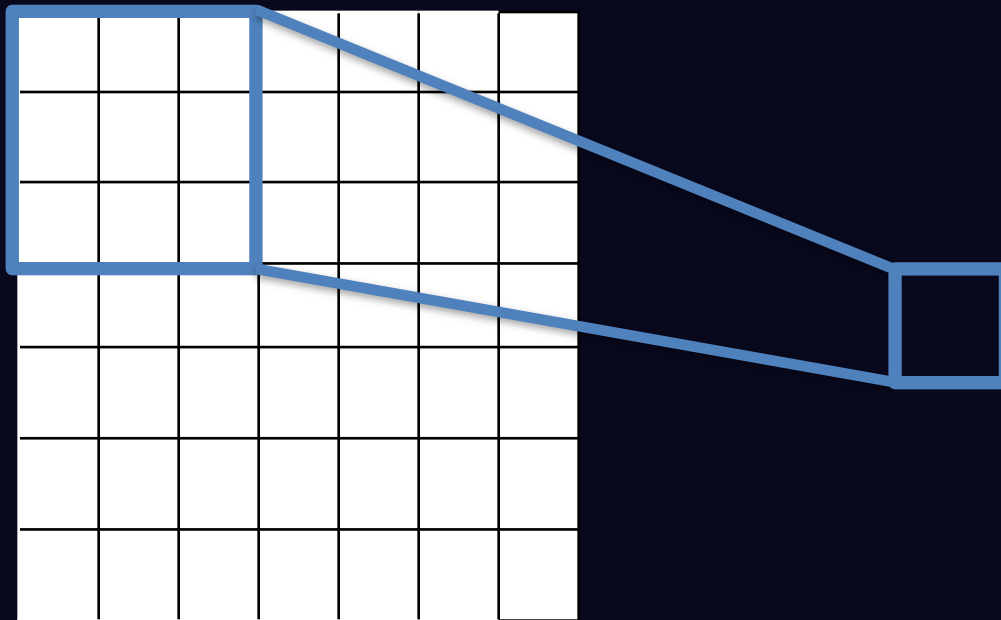
Slide filter over image



Convolution

Slide filter over image

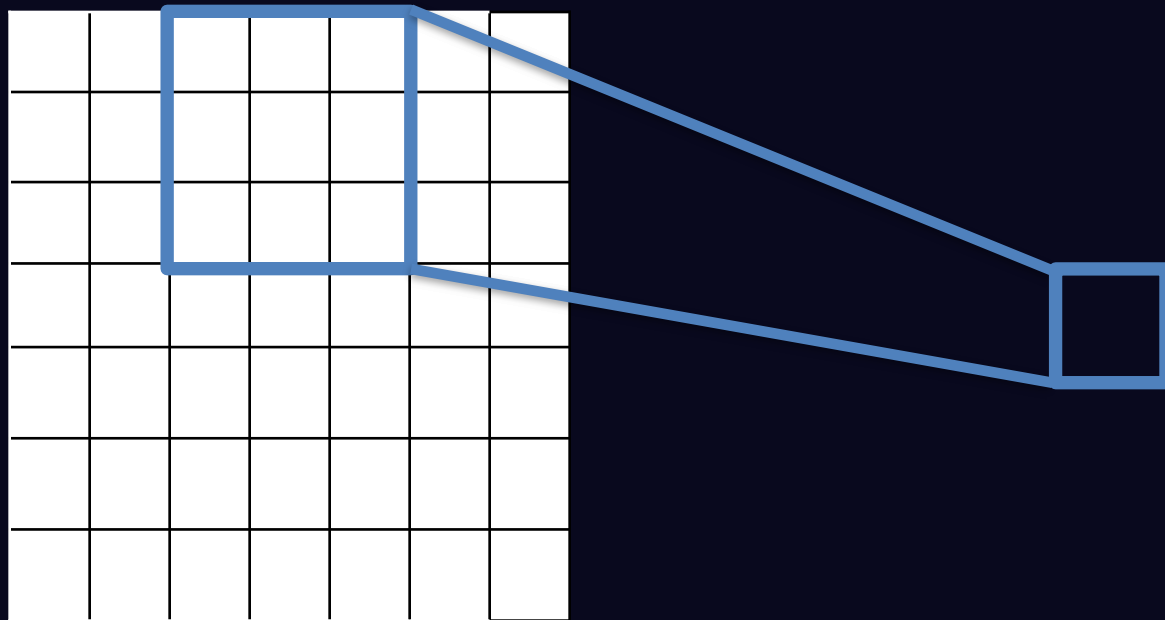
Stride 2



Convolution

Slide filter over image

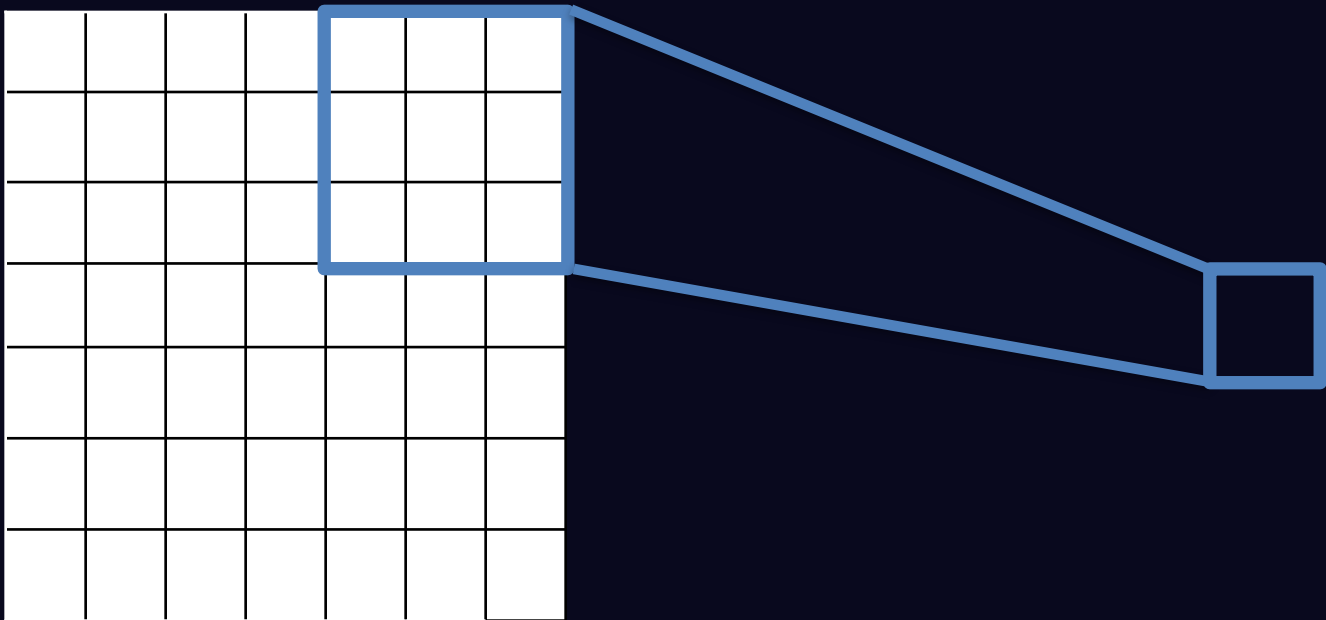
Stride 2



Convolution

Slide filter over image

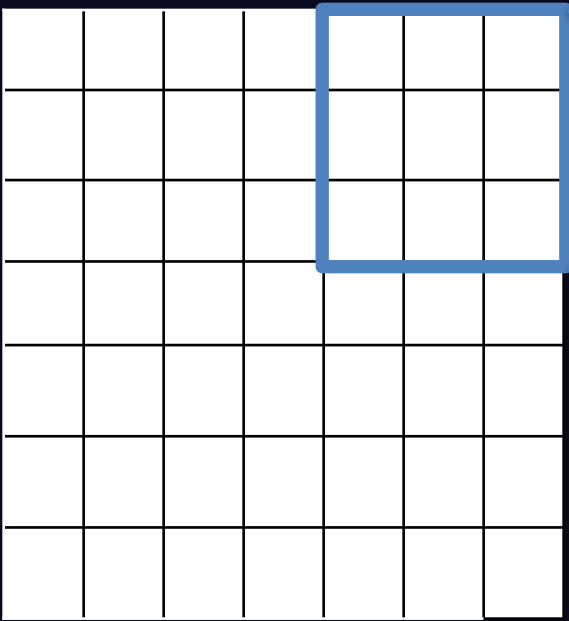
Stride 2



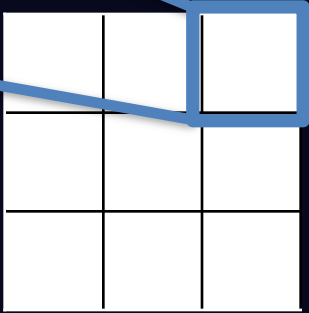
Convolution

Slide filter over image

Stride 2



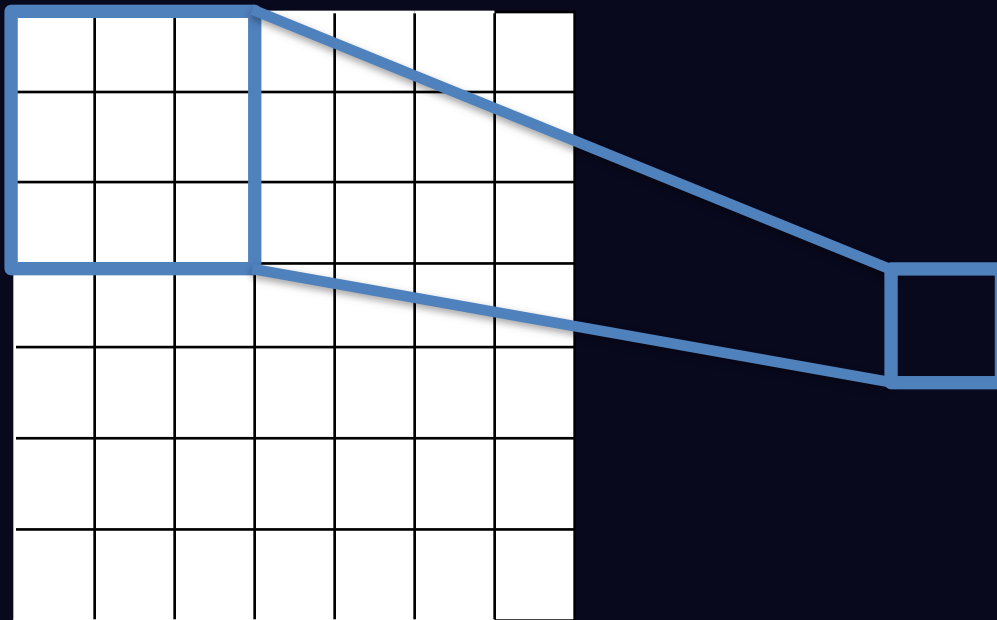
3x3 output



Convolution

Slide filter over image

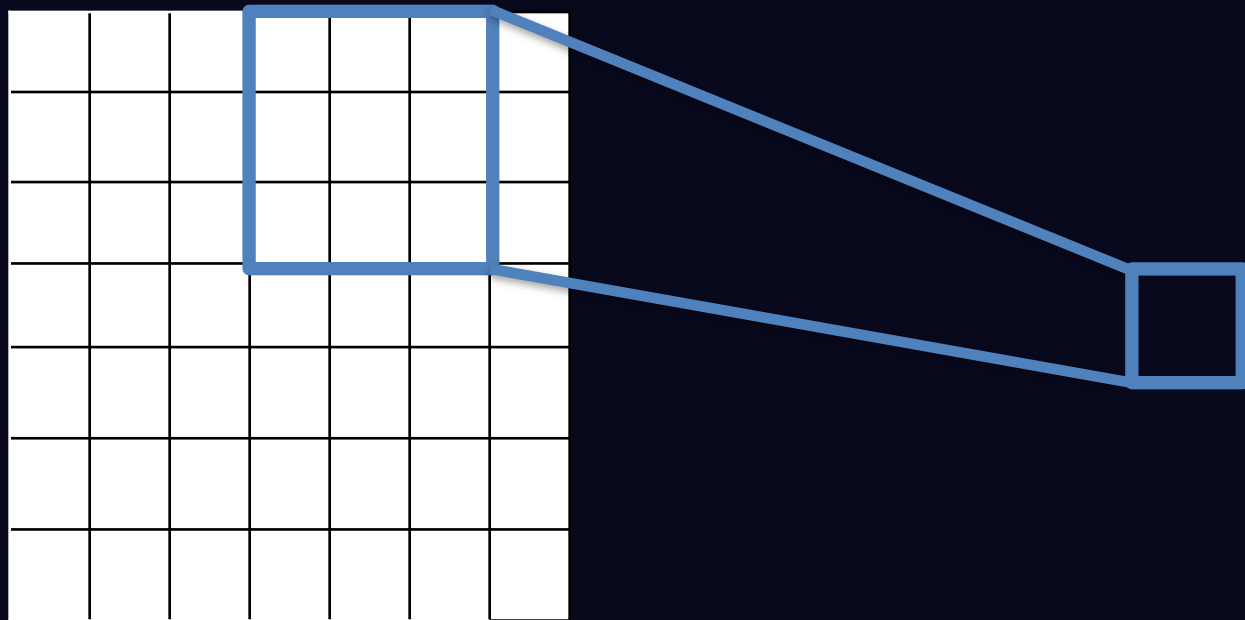
Stride 3



Convolution

Slide filter over image

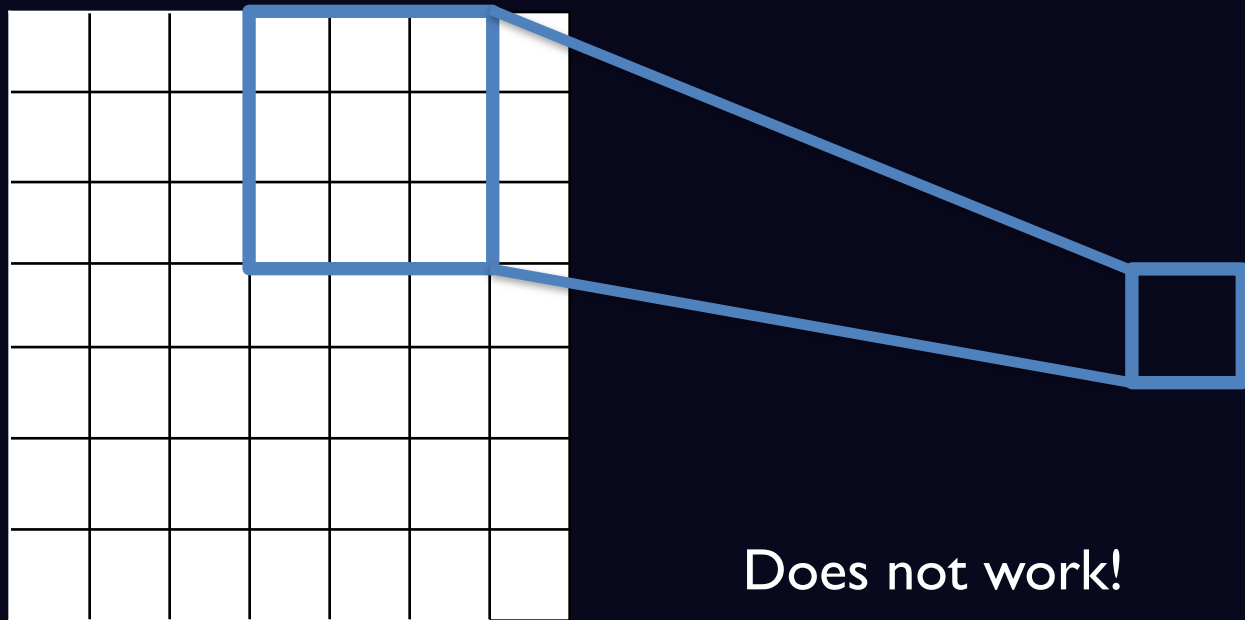
Stride 3



Convolution

Slide filter over image

Stride 3



Does not work!

⇒ Zero-pad the boundary.

Convolution Layer

Hyperparameters:

- Number of filters
- Filter size
- Stride
- Padding

CNN architecture components

- Convolution Layers
- Pooling Layers
- Fully Connected Layers
- Activation

Pooling Layer

- Coarse-grained spatial information
- No trainable parameters

Pooling Layer

- Coarse-grained spatial information
- No trainable parameters

Max Pool:

3	4		
6	3		

2x2 filter
Stride 2



Pooling Layer

- Coarse-grained spatial information
- No trainable parameters

Max Pool:

3	4		
6	3		

2x2 filter
Stride 2



6	

Pooling Layer

- Coarse-grained spatial information
- No trainable parameters

Max Pool:

3	4	7	6
6	3	6	3

2x2 filter
Stride 2



6	

Pooling Layer

- Coarse-grained spatial information
- No trainable parameters

Max Pool:

3	4	7	6
6	3	6	3

2x2 filter
Stride 2



6	7

Pooling Layer

Hyperparameters:

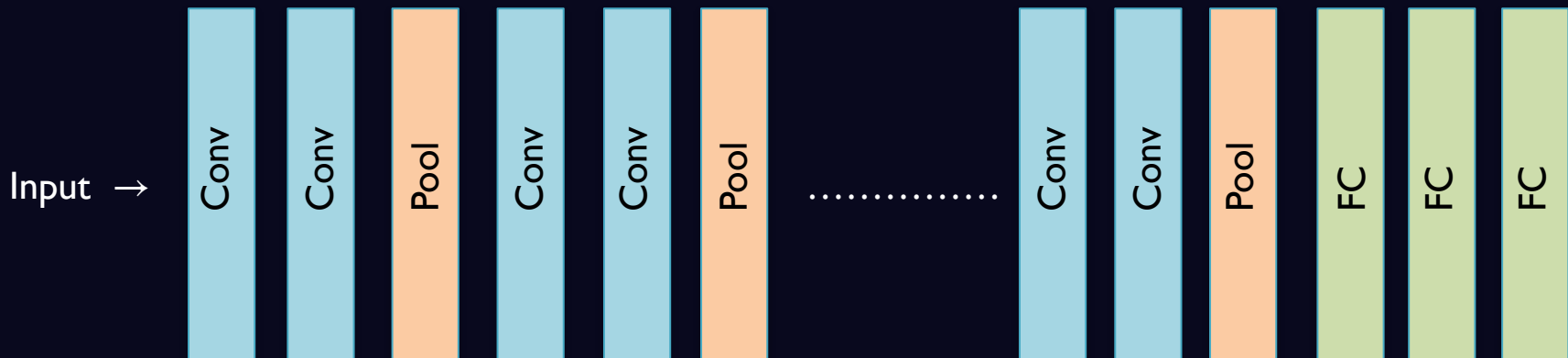
- Filter size
- Stride

CNN architecture components

- Convolution Layers
- Pooling Layers
- Fully Connected Layers
- Activation

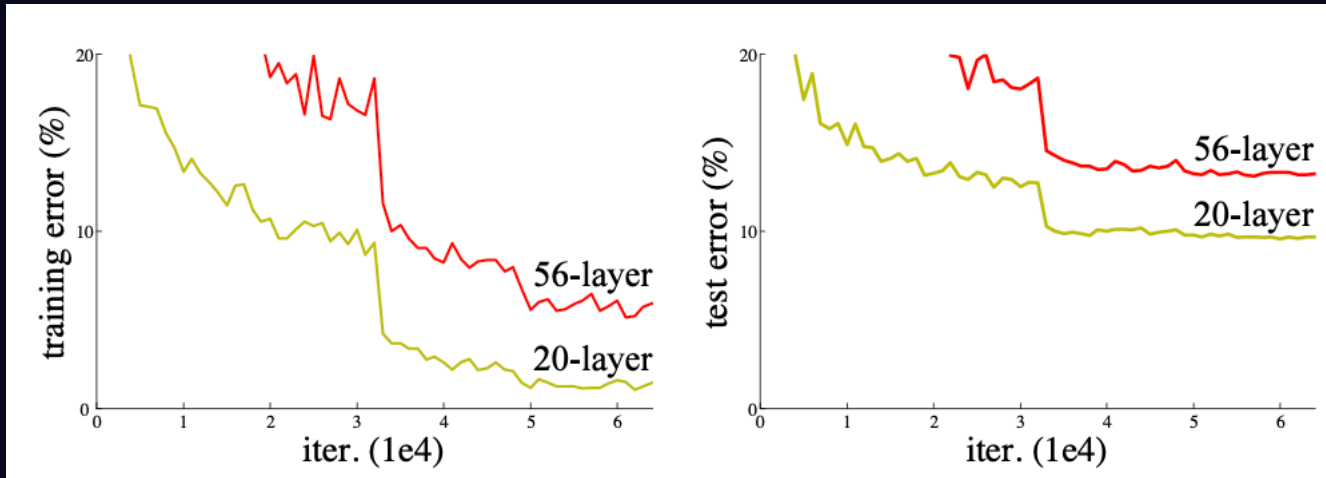
Example: VGG-Net

- Simonyan and Zisserman, ICLR 2015.
- Stack several convolutional layers (with activation) followed by a pooling layer.
- Final layers: fully connected.



Depth

CIFAR-10 data



He et al., 2015

Batch Normalization [Ioffe and Szegedy, 2015]

- Compute mean and standard deviations of the activations across mini batch.
- Use this to normalize the activations $\hat{z}$
- 2 learnable parameters: γ and β

$$\gamma \hat{z} + \beta$$

Batch Normalization [Ioffe and Szegedy, 2015]

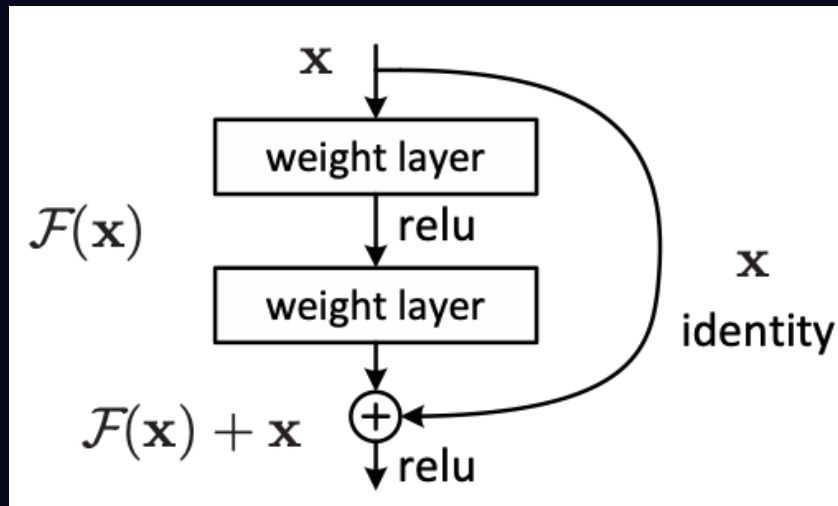
- Compute mean and standard deviations of the activations across mini batch.
- Use this to normalize the activations $\hat{z}$
- 2 learnable parameters: γ and β

$$\gamma \hat{z} + \beta$$

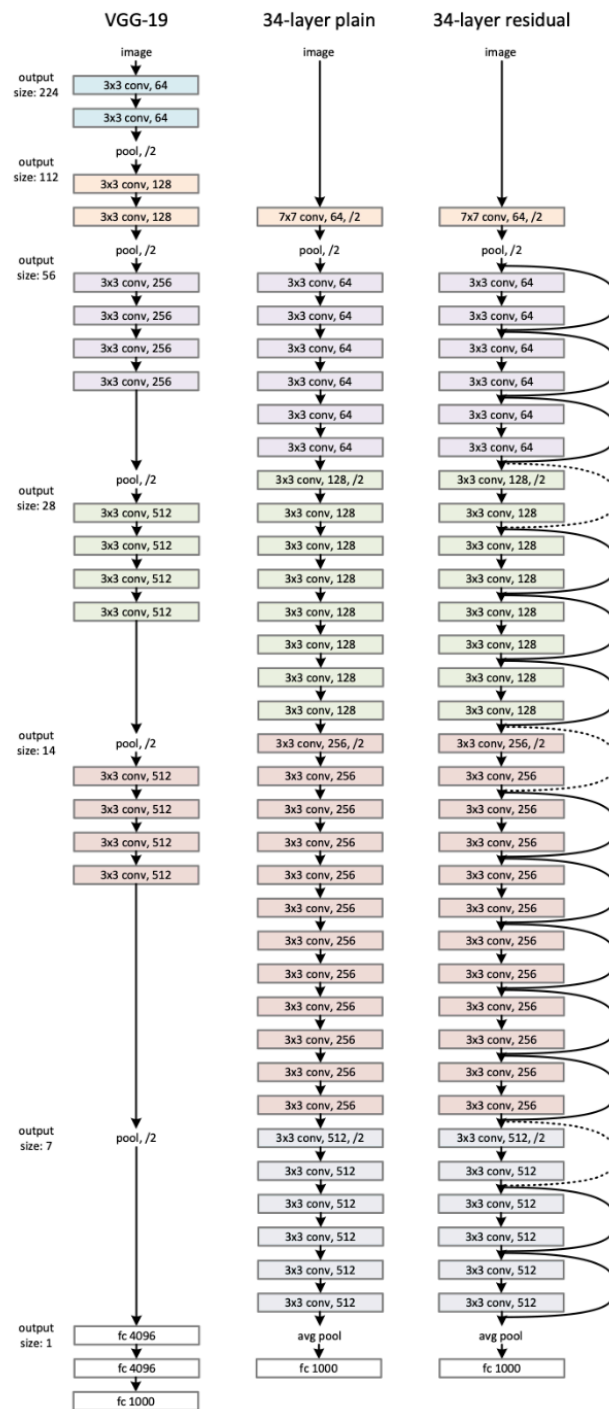
- Very successful!
- No rigorous analysis known.

Future lecture...!

Skip Connections



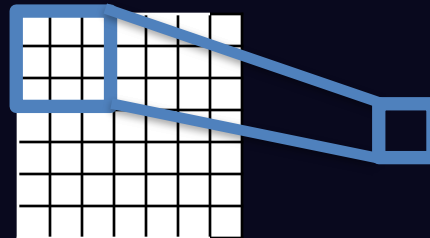
ResNet [He et al., 2015]
2015 state-of-the-art ImageNet competition



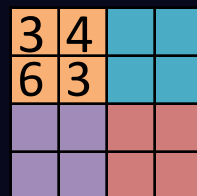
He et al., 2015

Summary: Convolutional Neural Networks

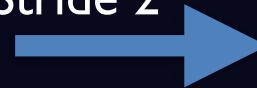
- Convolution Layers



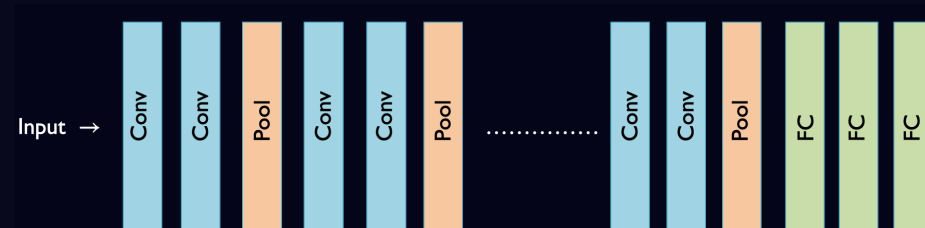
- Pooling Layers



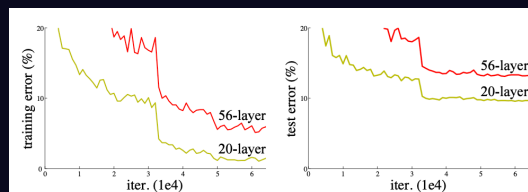
2x2 filter
Stride 2



- How building blocks fit together



- Depth



- ResNet architecture
- Batch Normalization

Questions?